# Bahir Dar University

# Bahir Dar Institute of Technology

# Faculty of Computing

**Requirement Analysis Document (RAD)**

**For**

**Microfinance and Saving Management System (MSMS)**

**Submitted to the faculty of computing in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science**


| Group Members | IDNumber |
|---|---|
| Bekalu Temesgen | 1505274 |
| Adino Aschalew | 1504923 |
| Abunu Mezgebu | 1504880 |

**Advisor: Elias M.**


**2018 E.C**

**Bahir Dar University,Bahir Dar Institute of Technology**

## Declaration

We declare that this project is our own work and has not been submitted to any other university for the award of a degree. All sources of information used in this document have been acknowledged.

| **Name** | **Signature** |
|---|---|
| Bekalu Temesgen | |
| Adino Aschalew | |
| Abunu Mezgebu | |

**Faculty:** Computing

**Program**: Computer Science

**Project Title**: Microfinance and Saving Management System (MSMS)

This is to certify that I have read this project and that in my supervision and the students' performance, it is fully adequate, in scope and quality, as a project for the degree of Bachelor of Science.

**Nameof Advisor : Mr Elias**

**Signature :**

| **Examining committee members** | **signature** | **Date** |
|---|---|---|
| 1. Examiner1 _____ | _____ | _____ |
| 2. Examiner2 _____ | _____ | _____ |

It is approved that this project has been written in compliance with the formatting rules laid down by the faculty

## Roles and Responsibilities of the Group Members

This section presents the distribution of responsibilities among group members. Each member participated in multiple activities to ensure shared understanding and collaboration.

| List of Tasks | Adino Ashalew | Bekalu Temesgen | Abunu Mezgebu |
|---|---|---|---|
| Project Planning & Management | | ✓ | |
| Literature Review | ✓ | ✓ | ✓ |
| Requirement Analysis & Documentation | ✓ | | ✓ |
| UI/UX Design & Frontend Development (React.js/React Native) | | ✓ | |
| Backend Development (Node.js/Database) | ✓ | | ✓ |
| Testing & Quality Assurance | ✓ | ✓ | ✓ |
| Final Documentation & Presentation | ✓ | ✓ | ✓ |

## Acknowledgment

First and foremost, we express our deepest gratitude to Almighty God, whose grace and blessings have enabled us to successfully complete our final year project.

We would like to express our sincere gratitude to Bahir Dar University, Bahir Dar Institute of Technology, Faculty of Computing, for providing us with the opportunity and resources to carry out this project.

We are also deeply grateful to our advisor Elias , for continuous guidance, constructive feedback, and encouragement throughout the project life-cycle.

## *List of Acronyms*

| Acronym | Expansion |
|---|---|
| API | Application Programming Interface |
| DBMS | Database Management System |
| ERD | Entity Relationship Diagram |
| JWT | JSON Web Token |
| HTTP | Hyper Text Transfer Protocol |
| HTTPS | Hyper Text Transfer Protocol Secure |
| MVC | Model View Controller |
| CRUD | Creat Read Update Delete |
| ORM | Object Relational Mapping |
| UI | User Interface |
| UX | User Experience |
| RDBMS | Relational Database Management System |
| UML | Unified Modeling Language |
| OOAD | Object Oriented Analysis and Design |
| ETB | Ethiopian Birr |
| REST | Representational State Transfer |
| JSON | Javascript Object Notation |
| SQL | Structured Query Language |
| FR | Functional Requirement |
| NFR | Non-Functional Requirement |
| SDLC | Software Development Life-Cycle |
| MSMS | Microfinance and Saving Management System |
| RAD | Requirement Analysis Documentation |
| CRC | Class Responsibility Collaboration |

## *List of Figures*


| Figure No. | Figure Caption | Page No. |
|---|---|---|
| Figure 2.1 | Use Case Diagram | 26 |
| Figure 2.2 | State Chart – User Account | 43 |
| Figure 2.3 | State Chart – Loan Lifecycle | 44 |
| Figure 2.4 | Activity – User Registration | 46 |
| Figure 2.5 | Activity – User Login | 47 |
| Figure 2.6 | Activity – Savings Deposit | 48 |
| Figure 2.7 | Activity – Savings Withdrawal | 48 |
| Figure 2.8 | Activity – Loan Application | 49 |
| Figure 2.9 | Activity – Loan Repayment | 49 |
| Figure 2.10 | Sequence – User Login | 50 |
| Figure 2.11 | Sequence – Member Registration | 51 |
| Figure 2.12 | Sequence – Savings Transaction | 52 |
| Figure 2.13 | Sequence – Loan Processing | 53 |
| Figure 2.14 | Sequence – Loan Repayment | 53 |

| | | |
|---|---|---|
| Figure 2.15 | Analysis Class Diagram | 54 |
| Figure 3.1 | System Architecture | 55 |
| Figure 3.2 | ER Diagram | 56 |
| Figure 3.3 | Deployment Diagram | 57 |
| Figure 3.4 | Login & Registration UI | 58 |
| Figure 3.5 | Admin Pages | 59 |
| Figure 3.6 | User Dashboard | 60 |

## *List of Tables*


| Table No. | Table Title | Page No. |
|---|---|---|
| Table 1.1 | Roles & Responsibilities | 2 |
| Table 1.2 | Acronyms & Abbreviations | 4 |
| Table 2.1 | Functional Req. – Member Mgmt | 20 |
| Table 2.2 | Functional Req. – Savings Mgmt | 21 |
| Table 2.3 | Functional Req. – Loan Mgmt | 22 |
| Table 2.4 | Functional Req. – Reporting | 23 |
| Table 2.5 | Business Rules Summary | 25 |
| Table 2.6 | Use Case – Manage Members | 27 |
| Table 2.7 | Use Case – Manage Savings | 28 |
| Table 2.8 | Use Case – Manage Loans | 29 |
| Table 2.9 | Use Case – Generate Reports | 29 |
| Table 2.10 | Use Case – Authentication | 30 |
| Table 2.11 | User Login Use Case | 31 |
| Table 3.1 | Database Tables & Attributes | 54 |
| Table 4.1 | Technology Stack | 62 |
| Table 5.1 | Test Cases – Login | 70 |
| Table 5.2 | Test Cases – Savings | 71 |
| Table 5.3 | Test Cases – Loan | 72 |

## Table of Contents


Acknowledgment ..... III
*List of Acronyms* ..... IV
Abstract ..... 1
Chapter One: Introduction ..... 2
1.1 Background ..... 2
1.2 Statement of the Problem ..... 3
1.3 Objectives ..... 4
1.3.1 General Objective ..... 4
1.3.2 Specific Objectives ..... 4
1.4 Methodology. ..... 4
1.4.1 Requirement Gathering Methods ..... 4
1.4.2 Analysis and Design Methodology ..... 5
1.4.3 Implementation Methodology ..... 6
1.5 Feasibility ..... 7
1.5.1 Economic Feasibility ..... 7
1.5.2 Technical Feasibility ..... 8
1.5.3 Schedule (Time) Feasibility ..... 8
1.5.4 Overall Feasibility Assessment ..... 9
1.6 Beneficiaries or Significance of the Project ..... 9
1.6.1 Benefits to System Administrators ..... 9
1.6.2 Benefits to Members ..... 9
1.6.3 Benefits to the Institution ..... 10
1.6.4 Academic and Social Significance ..... 10
1.6.5 Overall Significance ..... 10
1.7 Limitations of the Project ..... 11
1.8 Scope of the Project ..... 12
1.8.1 Functional Scope ..... 12
1.8.2 User Scope ..... 13
1.8.3 Operational Scope ..... 13
1.8.4 Out-of-Scope Features ..... 13
1.9 Organization of the Project ..... 13
Chapter Two: System Features ..... 15
2.1 The Existing System ..... 15
2.2 Proposed System ..... 16
2.2.1 Key Features of the Proposed System ..... 16
2.2.2 Benefits of the Proposed System ..... 18
2.2.3 Overview of System Operation ..... 18
2.3 Requirement Analysis ..... 19
2.3.1 Functional Requirements ..... 19
2.3.1 Functional Requirements ..... 19
2.3.2 System Use Case ..... 23
2.3.2.1 Use Case Diagram ..... 24

2.3.2.2 Use Case Documentation ····· 26
Use Case 1: Manage Members: ····· 26
Use Case 2: Manage Savings: ····· 26
Table:2.7 Use Case 2: Manage Savings: ····· 27
Use Case 3: Manage Loans: ····· 27
Table:2.8 Use Case 3: Manage Loans ····· 27
Use Case 4: Generate Reports: ····· 27
Table:2.9 Use Case 4: Generate Reports ····· 28
Use Case 5: User Authentication and Access Control: ····· 28
Table:2.10 Use Case 5: User Authentication and Access Control: ····· 28
User Login Usecase ····· 29
Table:2.11 User Login Usecase ····· 29
Deposit Savings ····· 30
Table:2.13 Deposit Savings ····· 30
2.3.3 Business Rule Documentation ····· 34
2.3.3.1 Key Business Rules for MSMS ····· 34
2.3.3.2 Notes on Business Rules ····· 35
Business Rules Table ····· 36
2.3.4 User Interface Prototype ····· 37
Purpose of the UI Prototype ····· 37
Key Features of the UI Prototype ····· 37
2.3.5 State Chart Diagram ····· 40
Fig :2.4 State Chart User Loan Diagram ····· 41
2.3.6 Activity Diagram ····· 42
Fig:2.6 Login Activity Diagram ····· 43
Fig: 2.7 Loan Activity Diagram ····· 44
Fig: 2.8 Withdrawal Saving Activity Diagram ····· 45
Fig: 2.9 Saving Deposit Activity Diagram ····· 45
Fig 2.10 Repay Loan Activity Diagram ····· 46
2.3.7 Sequence Diagram ····· 46
Fig :2.11 Login Sequence Diagram ····· 47
Fig 2.12 Registration Sequence Diagram ····· 47
Fig 2.13 Saving Sequence Diagram ····· 48
Fig 2.14 Withdraw Savings Sequence Diagram ····· 48
Fig : 2.15 Loan Sequence Diagram ····· 49
Fig :2.16 Repay Loan Sequence Diagram ····· 49
2.3.8 Analysis Class Model ····· 50
2.3.9 Logic Model ····· 51
♦ *For Login* ····· 51
♦ *Sign-Up* ····· 51
♦ *For Logout* ····· 52
2.Member Logic Model ····· 52
3.Admin Logic Model ····· 55
2.4 Non-Functional Requirements ····· 56
2.4.1 Performance ····· 57

2.4.2 Security ..... 57
2.4.3 Usability ..... 57
2.4.4 Reliability ..... 58
2.4.5 Maintainability and Scalability ..... 58
2.5 System Requirements ..... 58
2.5.1 Hardware Requirements ..... 58
1. CPU: ..... 58
2. RAM: ..... 59
3. Storage: ..... 59
4. Network & Connectivity: ..... 59
2.5.2 Software Requirements ..... 59
1. Operating System: ..... 59
2. Development Tools: ..... 59
3. Database: ..... 59
4. Version Control: ..... 59
5. Browser Requirements: ..... 59
6. Security Tools: ..... 60
7. Testing Tools: ..... 60
2.6 Key Abstraction with CRC Analysis ..... 60
Key Classes and CRC Details ..... 60
1. User ..... 60
2. Member ..... 60
3. Administrator ..... 60
4 Savings ..... 60
5 Loan ..... 61
6 Transaction ..... 61
7 ReportGenerator ..... 61
2.6.1 Conceptual Modeling: Class Diagram ..... 62
2.6.2 Identifying Change Cases ..... 63
Potential Change Cases ..... 63
1. New Financial Services: ..... 63
2. Integration with External Systems: ..... 63
3. User Role Expansion: ..... 63
4. Reporting Enhancements: ..... 63
5. Security Enhancements: ..... 63
6. Performance Scaling: ..... 63
2.6.3 User Interface Prototyping ..... 64
Fig: 3.6 User Dashboard(Users Home page) ..... 67
Fig: 3.7 User Saving page() ..... 68
Chapter Three: System Design ..... 69
Design Goals and Criteria ..... 69
3.1 Architectural Design ..... 70
3.1.1 System Architecture Overview ..... 71
Presentation Tier (Client Layer) ..... 71
Application Tier (Server Layer) ..... 71
Data Tier (Database Layer) ..... 71

Fig : 3.1 Application tier ····· 72
3.1.2 Component Modeling ····· 72
Component Interaction and Design Rationale ····· 73
Fig 3.2: Component Diagram ····· 74
3.1.3 Deployment Modeling ····· 74
Deployment Interaction and Design Considerations ····· 75
3.2 Detailed Design ····· 76
3.2.1 Design Class Model ····· 76
Class Relationships and Design Considerations ····· 78
3.2.2 Persistent Model (Database Schema) ····· 80
Entity–Relationship Diagram (ERD) ····· 80
Persistence Design Considerations ····· 88
3.3 User Interface Design ····· 88
3.3.1 Design Principles ····· 89
3.3.2 Navigation Map ····· 89
Member Portal Navigation ····· 89
Administrator Portal Navigation ····· 89
3.3.3 UI Mockups ····· 90
3.4 Access Control and Security ····· 90
3.4.1 Authentication Mechanism ····· 90
Authentication Workflow ····· 91
Security Benefits of JWT in MSMS ····· 91
3.4.2 Authorization (Role-Based Access Control – RBAC) ····· 91
Role Definition ····· 91
Authorization Matrix ····· 92
Authorization Enforcement ····· 92
3.4.3 Data Security and Privacy ····· 92
Password Protection and Credential Security ····· 92
Secure Communication ····· 92
Input Validation and Attack Prevention ····· 93
Transaction Integrity and Non-Repudiation ····· 93
Data Privacy and Access Isolation ····· 93
Security Design Significance for a Financial System ····· 93
*References* ····· 94
Appendices ····· 95
Appendix A: Interview and Questionnaire ····· 95
A.1: Interview Protocol for System Administrators ····· 95
Appendix B: Detailed Test Case Suite ····· 95
Table B.1: Test Cases for User Authentication (Login) ····· 95
Table B.2: Test Cases for Savings Deposit (FR2.1) ····· 96
Appendix C: Detailed Data Dictionary ····· 96
Table C.1: User (Member/Admin) ····· 96
Table C.2: SavingsAccount ····· 97
Table C.3: Transaction ····· 97

## Abstract

The **Microfinance and Saving Management System (MSMS)** project addresses the critical need for digital transformation within the established manual financial management structure of the Poly Campus community. The existing system, characterized by paper-based ledgers and manual calculations, suffers from severe inefficiencies, a high risk of human error, and a lack of real-time transparency, which collectively hinder financial inclusion and trust. This project details the design, development, and implementation of the MSMS, a robust, secure, and scalable web and mobile application. The system automates core functions such as deposits, fund access, micro-loan management, and financial reporting. By leveraging modern technologies like React.js, Node.js, React Native and a relational database, the MSMS provides a centralized, transparent platform accessible to all members and administrators. This document serves as the comprehensive Requirement Analysis Document, outlining the functional and non-functional requirements, detailed UML models, and system specifications necessary to deliver a high-quality, professional-grade financial management solution.

# Chapter One: Introduction

## 1.1 Background

Microfinance and saving systems play a vital role in promoting financial inclusion by providing basic financial services such as savings, credit, and loan facilities to individuals and groups who have limited access to formal banking institutions. These systems are particularly important in institutional and community-based environments where members rely on collective savings and micro-loans to address personal, social, and economic needs. Traditionally, many of these financial activities are managed using manual procedures, including paper-based records and handwritten ledgers.

At Bahir Dar University, Bahir Dar Institute of Technology, Poly Campus, a community-based microfinance and saving practice has been established to support staff members and service providers through regular savings contributions and access to small loans. This financial practice is designed to encourage a culture of saving, mutual support, and economic stability among members of the campus community. However, despite its importance, the current management of this system relies heavily on manual processes.
The existing manual approach involves recording member information, savings transactions, loan applications, approvals, and repayments using physical documents and registers. While this approach has served the community for many years, it has become increasingly inefficient as the number of members and financial transactions continues to grow. Manual record-keeping is time-consuming, prone to calculation and transcription errors, and lacks adequate mechanisms for data security, transparency, and real-time access to financial information.

With the rapid advancement of information and communication technologies, digital financial management systems have become a practical and effective solution for addressing the limitations of manual microfinance operations. Automated systems improve accuracy, reduce operational delays, enhance data security, and provide timely financial reports that support informed decision-making. Web-based and mobile-based applications, in particular, enable users to access financial services conveniently and securely.

In response to these challenges and technological opportunities, the **Microfinance and Saving Management System (MSMS)** is proposed as a digital solution to modernize the existing manual system at the Poly Campus community. The MSMS aims to automate savings and loan management processes, ensure data integrity, enhance transparency, and improve overall efficiency. By transforming the traditional manual system into a centralized digital platform, the project seeks to support sustainable financial management and strengthen trust among community members.

## 1.2 Statement of the Problem

The microfinance and saving practice at Bahir Dar University Poly Campus currently operates using a manual, paper-based system to manage member registration, savings contributions, loan applications, approvals, and repayments. Although this system has been in use for a long period, it no longer meets the growing operational and management needs of the community. As the number of members and financial transactions increases, the limitations of the existing system have become more evident and problematic.

One of the major problems of the current system is **inefficiency in transaction processing**. All financial activities are recorded manually, which consumes significant time and effort from administrators. Routine tasks such as calculating balances, tracking loan repayments, and preparing financial reports require repetitive manual work, resulting in delays and reduced service quality for members.

Another critical issue is the **high risk of human error**. Manual data entry and calculations are prone to mistakes, including incorrect balances, missing records, and inconsistent data across different registers. These errors can lead to financial discrepancies, disputes among members, and a loss of confidence in the management of the saving and loan system.

The existing system also suffers from a **lack of transparency and real-time access to information**. Members are unable to independently view their savings balances, loan status, or transaction history without directly contacting administrators. This dependency creates information gaps and limits accountability. Additionally, administrators face difficulties in generating accurate and timely financial reports due to the absence of an automated reporting mechanism.

**Data security and record preservation** present another serious challenge. Physical documents and ledgers are vulnerable to loss, damage, and unauthorized access. There is no reliable backup mechanism to protect critical financial data, which exposes the system to potential data loss and manipulation risks.

Furthermore, the manual system has **limited scalability and flexibility**. It cannot easily adapt to future growth, policy changes, or the introduction of new financial services. As a result, the system is unable to support efficient decision-making and sustainable financial management in the long term.

Due to these problems, the existing manual microfinance and saving system is inefficient, unreliable, and unsuitable for modern financial management requirements. Therefore, there is a clear need for a **digital Microfinance and Saving Management System (MSMS)** that can automate financial operations,

improve accuracy, enhance transparency, ensure data security, and support the effective management of savings and loan services within the campus community.

## 1.3 Objectives

### 1.3.1 General Objective

The general objective of the MSMS project is to design, develop, and implement a secure, scalable , and user-friendly digital platform that automates the core operations of the Poly Campus community's microfinance and savings management system, thereby achieving a state of full digital financial inclusion for its members.

### 1.3.2 Specific Objectives

1. To automate the entire member life-cycle, including registration, profile management, and role-based access control.

2. To implement a comprehensive **Savings Management Module** capable of real-time deposit tracking, withdrawal processing, and automated interest calculation.

3. To develop a robust **Loan Management Module** that supports a full workflow from application submission, through multi-stage approval, to automated disbursement and detailed repayment tracking.

4. To establish a **Reporting and Analytic Module** that provides administrators with on-demand, accurate financial statements, transaction summaries, and a comprehensive audit log.

5. To deliver a high-performance, cross-platform solution (web and mobile) that ensures maximum accessibility and a superior user experience for all community members.

6. To enhance data security and integrity by implementing industry-standard encryption, backup protocols, and a detailed, non-repudiable transaction history.

## 1.4 Methodology.

### 1.4.1 Requirement Gathering Methods

Requirement gathering is a critical phase in the development of any software system, as it ensures that the final product aligns with the needs and expectations of stakeholders. For the MSMS project, multiple requirement gathering techniques were employed to collect accurate, reliable, and comprehensive information from different perspectives. Using a combination of methods helped to reduce ambiguity and improve the completeness of the requirements.

- ***Interviews:*** were conducted with key stakeholders, including system administrators, committee members, and selected users of the existing microfinance and saving system. These interviews were both structured and semi-structured, allowing stakeholders to explain current operational procedures, challenges, and expectations in detail. Through interviews, valuable insights were gained regarding daily workflows, decision-making processes, and common problems associated with the manual system.

- ***Document analysis:*** was also used to understand the current system in depth. Existing records such as member registration forms, savings ledgers, loan application forms, repayment schedules, and financial reports were reviewed. This analysis helped identify the types of data currently managed, business rules followed by the institution, and reporting requirements. It also revealed inconsistencies and inefficiencies in record keeping that the proposed system needs to address.

- ***Observation:*** was employed to study real-time system usage and operational practices. By observing administrators while performing routine tasks such as recording transactions and calculating balances, the project team was able to understand actual workflows beyond what was documented or verbally explained. Observation helped identify hidden challenges, time-consuming tasks, and areas where automation would significantly improve efficiency.

Additionally, **stakeholder discussions and informal meetings** were held to validate collected requirements and resolve conflicting expectations. These discussions enabled collaborative decision-making and ensured that all major stakeholders shared a common understanding of the proposed system's scope and functionality. Feedback from these discussions was incorporated into the finalized requirement specification.

By combining interviews, document analysis, observation, and stakeholder discussions, the project team ensured that the system requirements were accurate, complete, and aligned with real operational needs.

### 1.4.2 Analysis and Design Methodology

After gathering requirements, an **object-oriented analysis and design (OOAD)** methodology was adopted to model and structure the MSMS. This methodology was chosen because it supports modularity, reusability, scalability, and maintainability, which are essential qualities for a financial management system.

During the **analysis phase**, the collected requirements were carefully examined and categorized into functional and non-functional requirements. System actors, use cases, and business processes were identified and documented. The focus of this

phase was to understand *what* the system should do without considering implementation details.

To represent the system behavior and structure, **Unified Modeling Language (UML)** diagrams were used extensively. UML provides standardized visual representations that improve communication between developers and stakeholders. The following UML diagrams were prepared during analysis and design:

- **Use Case Diagrams** to illustrate interactions between users and the system.
- **Use Case Descriptions** to explain system functionality in detail.
- **Class Diagrams** to model system entities, attributes, methods, and relationships.
- **Sequence Diagrams** to show object interactions over time.
- **Activity Diagrams** to represent workflows and business processes.
- **State Chart Diagrams** to model state changes of key system entities.

In the **design phase**, the analysis models were refined into detailed system designs. Logical and conceptual models were developed to define data structures, system components, and interactions. Emphasis was placed on ensuring consistency, minimizing redundancy, and supporting future system enhancements. User interface prototypes were also designed to ensure usability and ease of navigation.
This object-oriented approach ensures that the MSMS is well-structured, understandable, and adaptable to future changes.

### 1.4.3 Implementation Methodology

The implementation of the Microfinance and Saving Management System follows a **phased and modular development approach**. This approach allows the system to be developed, tested, and improved incrementally, reducing development risks and improving overall quality.

Modern **web and mobile development technologies** are selected to ensure accessibility, scalability, and platform independence. The system architecture follows a layered design that separates presentation, business logic, and data management components. This separation improves maintainability and supports future expansion.

During implementation, each module—such as user management, savings management, loan processing, repayment tracking, and reporting—will be developed independently and then integrated into the complete system. Regular testing will be performed at each stage, including unit testing, integration testing,

and user acceptance testing, to ensure that the system functions correctly and meets specified requirements.

Security considerations are incorporated throughout the implementation process. User authentication, authorization, and data protection mechanisms are applied to safeguard sensitive financial information. Backup and recovery strategies are also considered to ensure data reliability.

Finally, system deployment will be followed by user training and documentation to support effective system adoption. Feedback from users will be used to refine and improve the system during later stages of development.

## 1.5 Feasibility

Feasibility analysis is an essential activity in software project planning, as it evaluates whether a proposed system can be developed successfully within given constraints. This section assesses the feasibility of the Microfinance and Saving Management System (MSMS) from three major perspectives: **economic feasibility**, **technical feasibility**, and **schedule (time) feasibility**. The analysis demonstrates that the project is practical, achievable, and sustainable within the available resources.

### 1.5.1 Economic Feasibility

Economic feasibility examines whether the expected benefits of the project justify the costs involved in developing and operating the system. The MSMS project is economically feasible because it requires minimal financial investment while providing significant long-term benefits.

The system is developed as an academic final-year project using **open-source technologies**, which eliminates the need for expensive software licenses. Development tools, programming languages, databases, and frameworks used in the project are freely available. Additionally, existing hardware resources such as personal computers and institutional infrastructure are sufficient for development, testing, and deployment purposes.

From a benefit perspective, the MSMS significantly reduces operational costs associated with the manual system. Automation minimizes paper usage, printing costs, and physical storage requirements. It also reduces the workload on administrators by automating repetitive tasks such as calculations, record updates, and report generation. Over time, these efficiencies translate into cost savings and improved productivity.

Furthermore, improved accuracy and transparency reduce the risk of financial errors and disputes, which can have hidden economic costs. Therefore, the benefits of implementing the MSMS outweigh the limited development and operational costs, making the project economically viable.

### 1.5.2 Technical Feasibility

Technical feasibility evaluates whether the required technology, skills, and infrastructure are available to develop and operate the proposed system. The MSMS project is technically feasible due to the availability of suitable technologies and the technical competence of the development team.

The system is designed using widely adopted **web and mobile technologies**, supported by robust database management systems. These technologies are well-documented, stable, and suitable for developing secure financial management applications. The object-oriented design approach and UML-based modeling further enhance system clarity and maintainability.

The project team possesses the necessary academic background and practical skills in software development, database management, and system analysis, gained through coursework and practical laboratory sessions. In addition, access to online learning resources and development communities provides further technical support when needed.

The required hardware infrastructure, including computers, networking facilities, and internet access, is readily available within the institution. Since the system does not require specialized hardware or proprietary platforms, deployment and maintenance remain technically manageable.

Based on these factors, there are no significant technical barriers to the successful development and operation of the MSMS.

### 1.5.3 Schedule (Time) Feasibility

Schedule feasibility assesses whether the project can be completed within the allocated time frame. The MSMS project is time-feasible because it is planned and executed according to the academic calendar and final-year project timeline.

The project is divided into well-defined phases, including requirement analysis, system design, implementation, testing, documentation, and final submission. Each phase is assigned realistic time duration based on task complexity and available working hours. The use of a modular development approach allows different components to be developed and tested in parallel, improving time efficiency.

Potential delays are minimized through proper planning, regular progress monitoring, and continuous consultation with the project advisor. The scope of the system is clearly defined to avoid unnecessary feature expansion that could affect completion time.

Given the structured plan, manageable scope, and committed project team, the MSMS can be completed successfully within the scheduled time, making the project feasible from a time management perspective.

### 1.5.4 Overall Feasibility Assessment

Considering economic, technical, and schedule factors, the Microfinance and Saving Management System (MSMS) is **fully feasible**. The project can be developed using available resources, within the planned time frame, and at a reasonable cost, while delivering substantial benefits to the institution and its community.

## 1.6 Beneficiaries or Significance of the Project

The Microfinance and Saving Management System (MSMS) is designed to provide significant benefits to various stakeholders involved in the management and utilization of microfinance and saving services within the institution. The implementation of the system contributes not only to operational efficiency but also to transparency, accountability, and sustainable financial management. The primary beneficiaries of the project include system administrators, registered members, and the institution at large.

### 1.6.1 Benefits to System Administrators

System administrators are among the main beneficiaries of the MSMS. The automated system significantly reduces the workload associated with manual record keeping and repetitive administrative tasks. Activities such as member registration, savings entry, loan processing, repayment tracking, and report generation are handled electronically, saving time and effort.

The system provides administrators with accurate and up-to-date financial data, enabling better monitoring and control of savings and loan operations. Built-in reporting tools allow administrators to generate financial summaries, transaction histories, and performance reports quickly and reliably. This improves decision-making and supports effective financial planning.

Additionally, the system enhances data security and access control. Only authorized users can access sensitive information, reducing the risk of data manipulation or unauthorized disclosure. Overall, MSMS improves efficiency, accuracy, and accountability in administrative operations.

### 1.6.2 Benefits to Members

Members of the microfinance and saving system benefit significantly from the implementation of MSMS. The system improves transparency by allowing members to view their savings balances, loan status, and transaction history in a timely manner. This reduces dependency on administrators for information and increases trust in the financial management process.

The digital system ensures accurate recording of all transactions, minimizing errors that could negatively affect member accounts. Faster processing of savings and loan services improves user satisfaction and accessibility. Members also benefit from improved confidentiality, as personal and financial information is securely stored and protected within the system.

By enhancing convenience and reliability, MSMS encourages a stronger saving culture and responsible financial behavior among members.

### 1.6.3 Benefits to the Institution

The institution as a whole benefits from improved financial governance and operational efficiency. The availability of accurate and timely financial reports supports auditing, compliance, and strategic planning. The system reduces paper usage and physical storage needs, contributing to cost reduction and environmental sustainability.

The MSMS also enhances the institution's reputation by demonstrating the adoption of modern information systems and best practices in financial management. A reliable and transparent microfinance system strengthens institutional trust and supports long-term sustainability.

### 1.6.4 Academic and Social Significance

From an academic perspective, the project provides practical experience in applying software engineering principles, system analysis, and object-oriented design. It serves as a valuable learning platform for students and can be used as a reference for future academic projects.
Socially, the system promotes fairness, transparency, and financial inclusion within the community. By supporting efficient savings and loan management, the MSMS contributes to economic stability and mutual support among members.

### 1.6.5 Overall Significance

In summary, the Microfinance and Saving Management System has substantial significance for administrators, members, and the institution. By improving efficiency, transparency, security, and decision-making, the system addresses critical challenges of the existing manual process and supports sustainable financial management.

## 1.7 Limitations of the Project

Despite the expected benefits and feasibility of the Microfinance and Saving Management System (MSMS), the project is subject to certain limitations. These limitations arise mainly due to constraints related to time, scope, resources, and operational environment. Understanding these limitations helps set realistic expectations regarding the system's capabilities and provides a basis for future enhancements.

One of the primary limitations of the project is **time constraint**. As a final-year academic project, the development of MSMS must be completed within a fixed academic schedule. This limited time-frame restricts the ability to implement all possible features and conduct extensive long-term testing under real operational conditions. As a result, some advanced functionalities and optimizations may be deferred for future development phases.

Another significant limitation is **scope constraint**. The project focuses mainly on core microfinance and saving operations such as member registration, savings management, loan processing, repayment tracking, and basic reporting. Features such as integration with external banking systems, mobile money platforms, biometric authentication, and advanced analytic are outside the current project scope. These features, although valuable, require additional time, expertise, and institutional approval.

**Resource constraints** also affect the project. The development is carried out using limited hardware resources and relies primarily on open-source software tools. While these tools are sufficient for system development, they may not offer the same level of performance or scalability as enterprise-grade solutions. In addition, the project is developed by a small team of students, which limits the amount of parallel development and testing that can be performed.

The project is also limited by **data availability and user participation**. Since the system is developed based on sample or partial real data, access to complete historical financial records may be restricted due to privacy and policy considerations. Furthermore, user feedback during development may be limited, which can affect the refinement of certain system features.

Finally, the system's effectiveness depends on **user adoption and technological readiness**. Users who are not familiar with digital systems may require training to use the system effectively. Limited access to reliable internet connectivity or computing devices may also affect system usage in some situations.
Despite these limitations, the MSMS provides a strong foundation for automating microfinance and saving operations. The identified constraints can be addressed in

future versions of the system through additional development, resource allocation, and institutional support.

## 1.8 Scope of the Project

The scope of the Microfinance and Saving Management System (MSMS) defines the boundaries and extent of the system's functionality. It specifies the features, services, and operations that the system is designed to support, as well as what is intentionally excluded from the current project. Clearly defining the scope ensures that the project remains focused, manageable, and achievable within the available time and resources.

The MSMS is primarily designed to support the management of microfinance and saving activities within the institutional community. The system focuses on automating core financial operations that are currently handled manually, with the goal of improving efficiency, accuracy, transparency, and data security.

### 1.8.1 Functional Scope

The functional scope of the MSMS includes the following major components:

- ***Member-management:***
  The system supports the registration, updating, and management of member information. Each member has a unique profile that stores personal details, membership status, and account-related information. The system allows administrators to add new members, update existing records, and manage membership status in a structured and secure manner.

- ***Savings-management:***
  MSMS enables the recording and management of savings transactions. This includes periodic savings contributions, balance calculations, and transaction history tracking. The system automatically updates member balances after each transaction, reducing errors associated with manual calculations.

- ***Loan-management:***
  The system manages the complete loan life-cycle, including loan application, approval, disbursement, and monitoring. It records loan amounts, interest rates, repayment schedules, and outstanding balances. Administrators can track loan status and ensure compliance with institutional policies.

- ***Repayment-management:***
  MSMS supports the recording of loan repayments and automatically updates loan balances. The system tracks repayment history and identifies overdue or incomplete payments, helping administrators monitor loan performance effectively.

- ***Reporting--and--Information--Management:***
  The system generates financial and operational reports such as savings summaries, loan reports, repayment status reports, and member account statements. These reports support monitoring, auditing, and informed decision-making.

### 1.8.2 User Scope

The system is designed to be used by defined user groups, primarily administrators and registered members. Administrators have full access to system functionalities for managing financial operations, while members have limited access to view their personal savings and loan information. User roles and permissions are clearly defined to ensure security and proper system usage.

### 1.8.3 Operational Scope

The MSMS operates as a centralized digital system that stores and processes data in a secure database. It supports daily financial operations and provides real-time access to information for authorized users. The system is intended for use within the institutional environment and is designed to support future scalability and enhancement.

### 1.8.4 Out-of-Scope Features

The current version of the MSMS does not include integration with external banking systems, mobile money services, or third-party financial platforms. Advanced financial analytics, biometric authentication, and regulatory compliance automation are also beyond the scope of this project. These features may be considered for future development phases.

In summary, the scope of the MSMS includes member management, savings management, loan processing, repayment tracking, and reporting functionalities. The system focuses on automating essential microfinance operations while maintaining a clear and manageable project boundary.

## 1.9 Organization of the Project

The project is organized into several chapters, each addressing specific aspects of the development process. Below is a summary of the structure:

**Chapter--One:Introduction**

This chapter introduces the project, outlining its background, the problem it seeks to solve, and the objectives of the system. It includes the methodology used for requirements gathering, system analysis, design, and implementation. Additionally, the chapter discusses the feasibility of the project, its significance to the stakeholders, and the limitations and scope of the project.

**Chapter--Two:System-Features**

This chapter compares the existing system with the proposed system and analyzes

the functional and non-functional requirements of the application. It includes use case documentation, business rules, user interface prototypes, system diagrams (e.g., activity, sequence, class, state chart), and other related models. It also describes the system requirements in terms of hardware and software.

### Chapter--Three:System-Design-and-Architecture

This chapter provides a detailed description of the system architecture, design patterns, and database schema. It discusses the system's structure, the flow of data, and the relationships between various components, ensuring that the design meets both functional and non-functional requirements.

### Chapter--Four:Implementation

This chapter explains the development process, including the tools and technologies used to implement the system. It details the code structure, logic, and integration of different components such as the backend, user interface, and database.

### Chapter--Five:Testing-and-Evaluation

This chapter outlines the testing methodology, including test cases, results, and how the system was evaluated. It includes both functional testing to ensure the application meets requirements and performance testing to assess the system's reliability and scalability.

### Chapter--Six:Conclusion-and-Recommendations

This chapter summarizes the project's achievements, discusses the challenges faced during development, and suggests areas for future improvement. It also highlights the significance of the project and its potential impact on the target users.

## *References*

A comprehensive list of all sources cited throughout the project.

## Appendices

This section includes additional information, such as detailed code snippets, user guides, or supplementary data relevant to the project.

The organization ensures a logical flow from conceptualization to implementation, evaluation, and conclusion, providing a thorough understanding of the project's development.

# Chapter Two: System Features

## 2.1 The Existing System

The current microfinance and saving management system at BahirDar University Poly Campus operates entirely manually, relying on **paper-based records and handwritten difficulty** to manage all financial activities. This traditional system has been used for many years and has provided a basic mechanism for tracking member savings, loan applications, approvals, and repayments. However, as the number of members and financial transactions grows, the limitations of this manual approach have become increasingly apparent.

In the existing system, all **member registration** is done physically. Administrators collect personal information and create individual files for each member, which are stored in cabinets or physical storage spaces. These files serve as the primary record of membership and contain information such as names, identification numbers, contact details, and initial deposits. Any updates or changes to member information require manual editing of these records, which can lead to errors or inconsistencies.
**Savings management** is also completely manual. Members submit their contributions to administrators, who then record the transaction details—such as date, amount, and member identification—by hand in ledger books. The total savings balance for each member is calculated manually, often using calculators or spreadsheets. This process is labor-intensive and time-consuming, making it difficult to provide real-time account information to members.

Similarly, **loan management** follows a manual workflow. Members submit loan applications in paper form, which administrators review and approve based on predefined policies. Repayment schedules are written on paper, and loan balances are updated manually after each repayment. The system does not provide automated notifications for overdue loans or generate alerts for missed payments, which can lead to delays in identifying and addressing repayment issues.

**Reporting and financial analysis** in the existing system are particularly cumbersome. Administrators generate reports, such as summaries of total savings, outstanding loans, and transaction histories, by manually compiling data from multiple ledger pages. This not only increases the risk of errors but also requires considerable time and effort, delaying decision-making and reducing operational efficiency.
The existing system suffers from several key limitations:

- **High dependency on human effort:** All processes from data entry to calculations and report generation are performed manually, which is slow and error-prone.
- **Data inaccessibility:** Records are stored physically, making it difficult for multiple administrators or members to access information simultaneously.

- **Limited transparency:** Members cannot independently verify their savings or loan status and must rely entirely on administrators for information.
- **Security and preservation issues:** Physical ledgers are vulnerable to loss, damage, or unauthorized access, which puts financial data at risk.
- **Scalability constraints:** The system cannot efficiently handle growth in membership or the introduction of new financial services.

Despite these limitations, the manual system has served as a trusted financial mechanism for the campus community. Its reliance on interpersonal trust and strict administrative oversight has maintained operational continuity. However, to improve efficiency, accuracy, transparency, and security, it is essential to transition to a **digital, automated system**.

## 2.2 Proposed System

The proposed **Microfinance and Saving Management System (MSMS)** is a fully digital system designed to **automate and streamline all financial operations** currently managed manually at BahirDar University Poly Campus. Unlike the existing paper-based system, the MSMS leverages modern web and mobile technologies to improve efficiency, accuracy, transparency, and security in managing member savings, loans, and reporting processes.

### 2.2.1 Key Features of the Proposed System

- ***Centralized Database Management:***

  All financial and member data is stored in a centralized, secure database. This ensures consistency, prevents data duplication, and allows authorized users to access real-time information. Centralized storage eliminates the inefficiencies and risks associated with maintaining multiple physical ledgers.

- ***Automated-Savings-Management:***

  The system allows members to make savings contributions digitally, either via the web application or mobile interface. Savings transactions are automatically recorded, and the member's balance is updated in real-time. Automatic calculation of interest ensures that members' savings grow accurately without manual intervention.

- ***Loan-Management-and-Processing:***

  MSMS manages the full lifecycle of loans, from application submission to approval, disbursement, and repayment tracking. Loan eligibility checks,

repayment schedules, and overdue notifications are automated, reducing administrative workload and minimizing errors. Administrators can efficiently review pending applications and approve or reject them based on predefined rules.

- ***Role-Based-Access-Control:***

  The system provides different access levels for administrators and members. Administrators have full access to all system functionalities, including transaction management, reporting, and user management, while members can access their personal account details, view balances, and track loan repayments. This ensures data security and restricts unauthorized access.

- ***Reporting-and-Analytic:***

  MSMS includes built-in reporting tools that generate comprehensive financial statements, summaries, and audit logs. Reports can be generated on-demand, exported in multiple formats, and used for auditing, decision-making, and monitoring system performance. Automated reporting eliminates manual compilation errors and provides real-time insights.

- ***User-Friendly-Interface:***

  The system provides an intuitive, responsive, and accessible user interface. The web interface is optimized for administrators, while the mobile interface caters to members, allowing convenient access to financial services anytime and anywhere. Notifications and alerts guide users and improve adherence to repayment schedules.

- ***Data-Security-and-Backup:***

  MSMS implements advanced security measures such as encrypted data storage, secure login, and backup mechanisms to protect sensitive financial information. Data integrity is ensured through validation rules and automatic audit trails for all transactions.

### 2.2.2 Benefits of the Proposed System

The transition from a manual system to MSMS provides several advantages:

- **Increased Efficiency:** Automated workflows significantly reduce time spent on routine administrative tasks.
- **Improved Accuracy:** Automatic calculations for balances, interest, and repayments minimize human errors.
- **Enhanced Transparency:** Members can view their account information in real-time, fostering trust and confidence in the system.
- **Better Data Security:** Digital storage and role-based access reduce risks of data loss, tampering, or unauthorized access.
- **Scalability:** The system can easily accommodate growth in membership and financial services without additional administrative burden.
- **Audit-ability:** The system maintains detailed logs of all operations, supporting financial audits and accountability.

### 2.2.3 Overview of System Operation

The proposed MSMS operates in a **three-tier architecture** consisting of:

1. **Presentation Layer:** Provides user interaction through web and mobile interfaces.
2. **Business Logic Layer:** Handles transaction processing, savings and loan calculations, and enforcement of business rules.
3. **Data Layer:** Stores all member, savings, loan, and transaction data securely in a relational database.

This architecture ensures modularity, maintainability, and ease of future enhancement. The system's workflow is designed to mirror actual financial operations while automating repetitive tasks, allowing administrators and members to focus on decision-making rather than record-keeping.

In summary, the proposed MSMS is a **digital, automated, and secure system** that transforms the traditional manual processes into an efficient and reliable platform. It addresses the limitations of the existing system by reducing errors, improving

transparency, enhancing security, and enabling scalable financial management. The system forms the foundation for a modern, community-based microfinance solution that supports both operational efficiency and stakeholder satisfaction.

## 2.3 Requirement Analysis

### 2.3.1 Functional Requirements

Requirement analysis is a critical phase in system development that defines the functional and non-functional needs of the system. It ensures that the Microfinance and Saving Management System (MSMS) meets the expectations of stakeholders and supports the operational objectives of the campus microfinance program.
Requirements are classified into **functional** and **non-functional** categories:

1. **Functional requirements** describe the specific services and operations the system must perform.

2. **Non-functional requirements** define constraints on the system's operation, such as performance, security, usability, and reliability.

## 2.3.1 Functional Requirements

Functional requirements specify the **actions, tasks, and behaviors** the system must perform. They describe the interaction between users (members and administrators) and the system, ensuring that all financial operations, reporting, and user management processes are properly supported. Functional requirements are typically captured per system module, and each requirement includes an identifier, description, priority, and source.

The MSMS functional requirements are organized into the following modules:

#### *2.3.1.1 Member Management Module*

| ID | Requirement Description | Priority | Source |
|---|---|---|---|
| FR1.1 | The system MUST allow an administrator to create a new member profile with full personal information. | High | Stakeholder Interview |
| FR1.2 | The system MUST enforce unique identification for each member (e.g., ID number). | High | Business Rule |
| FR1.3 | Administrators MUST be able to update member information and account status. | High | Stakeholder Interview |

| | | | |
|---|---|---|---|
| FR1.4 | Members MUST be able to view and update non-critical personal information. | Medium | User Request |
| FR1.5 | Each member profile MUST be automatically linked to a savings account. | High | Business Rule |

**Table:2.1 Member Management Module**

### *2.3.1.2 Savings Management Module*

| ID | Requirement Description | Priority | Source |
|---|---|---|---|
| FR2.1 | The system MUST allow administrators to record member deposits, including amount and date. | High | Stakeholder Interview |
| FR2.2 | The system MUST allow administrators to process member withdrawals. | High | Stakeholder Interview |
| FR2.3 | The system MUST update member balances automatically after each transaction. | Critical | Business Rule |
| FR2.4 | The system MUST prevent withdrawals exceeding available balance. | Critical | Business Rule |
| FR2.5 | Members MUST be able to view their detailed savings transaction history. | High | User Request |
| FR2.6 | The system MUST calculate and post interest automatically on active accounts. | High | Business Rule |

**Table:2.2 Savings Management Module**

### *2.3.1.3 Loan Management Module*

| ID | Requirement Description | Priority | Source |
|---|---|---|---|
| FR3.1 | Members MUST be able to submit loan applications with requested amount, purpose, and repayment period. | High | User Request |
| FR3.2 | The system MUST check loan eligibility based on predefined criteria. | Critical | Business Rule |
| FR3.3 | Administrators MUST have a dashboard to review pending loan applications. | High | Stakeholder Interview |
| FR3.4 | Administrators MUST approve or reject loans with mandatory justification. | High | Stakeholder Interview |
| FR3.5 | Upon approval, the system MUST generate a repayment schedule automatically. | Critical | Business Rule |
| FR3.6 | The system MUST record loan disbursement and update member's outstanding balance. | Critical | Business Rule |
| FR3.7 | Administrators MUST record repayments, which are applied automatically to interest and principal. | High | Stakeholder Interview |
| FR3.8 | The system MUST flag overdue loans if repayments are missed beyond allowed period. | High | Business Rule |
| FR3.9 | Members MUST be able to view detailed current and past loan information. | High | User Request |

**Table:2.3 Loan Management Module**

### *2.3.1.4 Reporting and Analytics Module*

| ID | Requirement Description | Priority | Source |
|---|---|---|---|
| FR4.1 | Administrators MUST generate transaction summary reports for specified date ranges. | High | Stakeholder Interview |
| FR4.2 | Administrators MUST generate outstanding loans | High | Stakeholder Interview |

| | | | |
|---|---|---|---|
| | reports with status details. | | |
| FR4.3 | Administrators MUST generate member account statements. | High | Stakeholder Interview |
| FR4.4 | Reports MUST be exportable in PDF and CSV formats. | Medium | User Request |
| FR4.5 | The system MUST maintain a non-editable audit log for all administrative actions. | Critical | Business Rule |

**Table:2.4 Reporting and Analytics Module**

Functional requirements provide a **detailed description of what the system must do**. They form the foundation for system design, implementation, and testing. By organizing requirements into modules, MSMS ensures clarity, traceability, and alignment with stakeholder expectations. Each requirement is linked to specific actors, processes, and business rules, which will be further mapped in **use case specifications and UML diagrams** in the following sections.

| FR ID | Functional Requirement Description |
|---|---|
| **FR-1** | The system shall allow an administrator to register, update, and deactivate member accounts. |
| **FR-2** | The system shall authenticate users using a secure login mechanism based on username and password. |
| **FR-3** | The system shall authorize users based on their assigned roles and permissions. |
| **FR-4** | The system shall allow members to deposit savings into their accounts. |
| **FR-5** | The system shall allow members to request savings withdrawals subject to approval rules. |
| **FR-6** | The system shall automatically update member savings balances after each transaction. |
| **FR-7** | The system shall record all financial transactions with date, amount, and transaction type. |
| **FR-8** | The system shall allow members to submit loan applications electronically. |
| **FR-9** | The system shall validate loan eligibility based on predefined business rules. |

| | |
|---|---|
| **FR-10** | The system shall allow administrators to approve or reject loan applications. |
| **FR-11** | The system shall generate a loan repayment schedule upon loan approval. |
| **FR-12** | The system shall allow members to make loan repayments. |
| **FR-13** | The system shall update loan balances automatically after each repayment. |
| **FR-14** | The system shall maintain the status of loans (pending, approved, active, completed). |
| **FR-15** | The system shall generate financial reports for savings, loans, and repayments. |
| **FR-16** | The system shall allow administrators to view individual member transaction histories. |
| **FR-17** | The system shall support search and filtering of members and transactions. |
| **FR-18** | The system shall allow administrators to manage user roles and access privileges. |
| **FR-19** | The system shall notify users of important system events such as loan approval or rejection. |
| **FR-20** | The system shall maintain an audit log of critical system activities. |

**Table:2.5** functional requirement

### 2.3.2 System Use Case

System use cases describe the **interactions between users (actors) and the system** to accomplish specific goals. They capture how the system should behave under different scenarios and define the responsibilities of each actor. For the MSMS, the primary actors include:

- **Administrator:** Responsible for managing members, processing savings and loans, generating reports, and maintaining system integrity.
- **Member:** Can view personal account information, submit loan applications, track savings, and monitor loan repayment schedules.

Use case analysis helps identify functional requirements, clarify system boundaries, and serve as the basis for detailed design and testing.

### 2.3.2.1 Use Case Diagram

A **use case diagram** visually represents the system's actors, use cases, and their interactions. It provides a high-level overview of the functionality of the MSMS. Key use cases include:

- **Manage Members:** Create, update, and maintain member records.
- **Manage Savings:** Record deposits and withdrawals, calculate balances, and update transaction history.
- **Manage Loans:** Process loan applications, approve/reject requests, generate repayment schedules, and record repayments.
- **Generate Reports:** Create savings reports, loan reports, member statements, and export data.
- **User Access & Authentication:** Ensure secure login and access control for both administrators and members.

# Use Case Figure

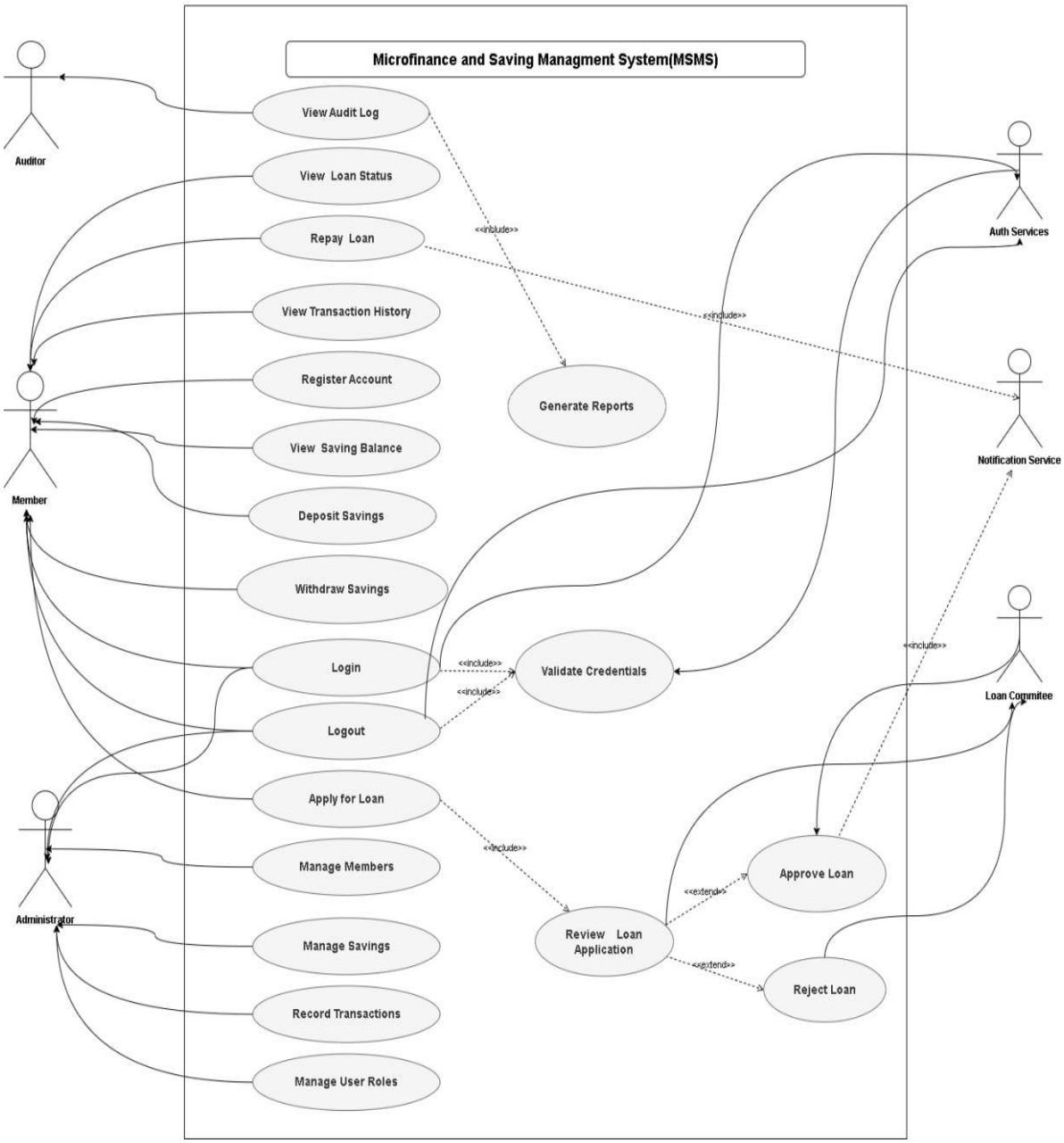



**Fig :2.1** Use Case

### 2.3.2.2 Use Case Documentation

Use case documentation provides a **detailed description of each system use case**, specifying actors, preconditions, triggers, main flow of events, alternative flows, and post-conditions. This ensures clarity in understanding how the MSMS will operate and serves as a foundation for system design, implementation, and testing.
Below are the key use cases for the MSMS:

### Use Case 1: Manage Members:

| **Attribute** | **Description** |
|---|---|
| Use Case ID | UC1 |
| Use Case Name | Manage Members |
| Actor | Administrator |
| Preconditions | Administrator is logged in and has appropriate access rights. |
| Trigger | Administrator selects the "Member Management" module. |
| Main Flow | 1.Administrator-adds a new member.<br>2.System-validates member information.<br>3.System creates member profile and links it to a savings account.<br>4.Administrator can update or delete member information.<br>5. System confirms action completion. |
| Alternative Flow | If mandatory member details are missing or invalid, system prompts for corrections. |

**Table:2.6** Use Case 1: Manage Members:

### Use Case 2: Manage Savings:

| **Attribute** | **Description** |
|---|---|
| Use Case ID | UC2 |
| Use Case Name | Manage Savings |
| Actor | Administrator, Member |
| Preconditions | Member account exists; user is authenticated. |
| Trigger | User selects "Deposit" or "Withdrawal" function. |

| | |
|---|---|
| Main Flow | 1. User enters savings transaction details.<br>2. System validates the transaction (e.g., withdrawal does not exceed balance).<br>3. System updates the member's balance and transaction history.<br>4. System confirms transaction completion. |
| Alternative Flow | For invalid input (e.g., insufficient funds), system displays an error message and rejects the transaction. |
| Postconditions | Savings transaction is accurately recorded, and account balance is updated. |

**Table:2.7** Use Case 2: Manage Savings:

## Use Case 3: Manage Loans:

| **Attribute** | **Description** |
|---|---|
| Use Case ID | UC3 |
| Use Case Name | Manage Loans |
| Actor | Administrator, Member |
| Preconditions | Member account exists; user is authenticated. |
| Trigger | Member submits loan application. |
| Main Flow | 1. Member submits a loan application.<br>2. System checks eligibility based on predefined rules.<br>3. Administrator reviews and approves/rejects the loan.<br>4. System generates repayment schedule for approved loans.<br>5. Member makes repayments, and the system updates outstanding balances.<br>6. System flags overdue loans. |
| Alternative Flow | If the member is ineligible or application is incomplete, system rejects the loan and notifies the member. |
| Postconditions | Loan is approved/rejected, and all repayment information is recorded accurately. |

**Table:2.8** Use Case 3: Manage Loans

## Use Case 4: Generate Reports:

| **Attribute** | **Description** |
|---|---|
| Use Case ID | UC4 |
| Use Case Name | Generate Reports |
| Actor | Administrator |

| | |
|---|---|
| Preconditions | Administrator is authenticated and has access to reporting module. |
| Trigger | Administrator requests a report. |
| Main Flow | 1. Administrator selects type of report (e.g., savings summary, loan status, member statements). 2. System retrieves and processes relevant data. 3. System generates the report and displays or exports it in the chosen format (PDF, CSV). |
| Alternative Flow | If data is incomplete or unavailable, system notifies the administrator and generates a partial report. |
| Postconditions | Report is successfully generated and available for review or export. |

**Table:2.9** Use Case 4: Generate Reports

## Use Case 5: User Authentication and Access Control:

| **Attribute** | **Description** |
|---|---|
| Use Case ID | UC5 |
| Use Case Name | User Authentication and Access Control |
| Actor | Administrator, Member |
| Preconditions | User has a registered account. |
| Trigger | User attempts to log in. |
| Main Flow | 1. User enters login credentials.<br>2. System validates credentials.<br>3. If credentials are correct, system grants access based on role.<br>4. User accesses allowed modules. |
| Alternative Flow | If credentials are incorrect, system denies access and displays an error message. |
| Postconditions | User is successfully authenticated and granted access according to role, or denied access if invalid credentials. |

**Table:2.10** Use Case 5: User Authentication and Access Control:

The use case documentation ensures that each functional requirement is **clearly mapped to system actions and actors**, providing detailed operational guidelines. These descriptions will later support the creation of UML diagrams such as **sequence diagrams, activity diagrams, and class diagrams**, making the system implementation and testing process precise and traceable.

## User Login Usecase

| Field | Description |
|---|---|
| Use Case ID | UC-01 |
| Use Case Name | User Login |
| Primary Actor | System User (Administrator, Staff, Member) |
| Secondary Actor | Authentication Service |
| Description | Allows a registered user to log into the MSMS using valid credentials |
| Preconditions | User is registered and account is active |
| Main Flow | 1. User opens login page<br>2. User enters username and password<br>3. System validates credentials<br>4. System identifies user role<br>5. System redirects to dashboard |
| Alternative Flow | Invalid credentials → system displays error and allows retry |
| Exception Flow | Account locked or inactive → access denied and user notified |

**Table:2.11** User Login Usecase

## Register Member Usecase

| Field | Description |
|---|---|
| Use Case ID | UC-02 |
| Use Case Name | Register Member |
| Primary Actor | Administrator |
| Description | Allows administrator to register a new member |
| Preconditions | Administrator is authenticated; member data is available |
| Postconditions | Member account is created with a unique ID |
| Main Flow | 1. Admin selects “Register Member”<br>2. System displays form<br>3. Admin enters member details<br>4. System validates data |

| | |
|---|---|
| | 5. System saves record<br>6. Confirmation displayed |
| Alternative Flow | Missing required fields → system prompts for completion |

**Table:2.12** Register Member Usecase

## Deposit Savings

| Field | Description |
|---|---|
| Use Case ID | UC-03 |
| Use Case Name | Deposit Savings |
| Primary Actor | Member |
| Secondary Actor | Administrator |
| Description | Records savings deposits into a member's account |
| Preconditions | Member is registered and account is active |
| Postconditions | Savings balance updated and transaction recorded |
| Main Flow | 1. Member requests deposit<br>2. Admin verifies request<br>3. System records deposit<br>4. Balance updated<br>5. Receipt generated |
| Alternative Flow | Invalid deposit amount → transaction rejected |

**Table:2.13** Deposit Savings

## Withdraw Savings

| Field | Description |
|---|---|
| Use Case ID | UC-04 |
| Use Case Name | Withdraw Savings |
| Primary Actor | Member |
| Secondary Actor | Administrator |
| Description | Handles withdrawal of savings from member account |
| Preconditions | Account is active and has sufficient balance |
| Postconditions | Balance reduced and transaction recorded |

| | |
|---|---|
| Main Flow | 1. Member submits withdrawal request<br>2. System checks balance<br>3. Admin approves request<br>4. System updates balance<br>5. Transaction recorded |
| Exception Flow | Insufficient balance → request rejected |

**Table:2.14** Withdraw Savings

## Apply for Loan

| **Field** | **Description** |
|---|---|
| Use Case ID | UC-05 |
| Use Case Name | Apply for Loan |
| Primary Actor | Member |
| Secondary Actor | Administrator |
| Description | Allows a member to apply for a loan |
| Preconditions | Member is eligible and meets savings requirements |
| Postconditions | Loan application recorded as pending |
| Main Flow | 1. Member submits loan application<br>2. System validates eligibility<br>3. Admin reviews application<br>4. System records request |
| Alternative Flow | Eligibility not met → application rejected |

**Table:2.15** Apply for Loan

## Approve Loan

| Field | Description |
|---|---|
| Use Case ID | UC-06 |
| Use Case Name | Approve Loan |
| Primary Actor | Administrator |
| Description | Allows administrator to approve or reject loans |
| Preconditions | Loan application exists; admin is authenticated |
| Postconditions | Loan status updated and member notified |
| Main Flow | 1. Admin reviews loan requests<br>2. Admin approves or rejects loan<br>3. System updates status<br>4. Member notified |

**Table:2.16** Approve Loan

## Repay Loan

| Field | Description |
|---|---|
| Use Case ID | UC-07 |
| Use Case Name | Repay Loan |
| Primary Actor | Member |
| Secondary Actor | Administrator |
| Description | Manages repayment of approved loans |
| Preconditions | Loan is active and repayment amount is valid |
| Postconditions | Loan balance reduced and repayment recorded |
| Main Flow | 1. Member initiates repayment<br>2. System calculates remaining balance<br>3. Admin confirms payment<br>4. System updates loan record |

**Table:2.17** Repay Loan

## Generate Financial Reports

| Field | Description |
|---|---|
| Use Case ID | UC-08 |
| Use Case Name | Generate Financial Reports |
| Primary Actor | Administrator |
| Description | Generates financial and operational reports |
| Preconditions | Relevant data exists in the system |
| Postconditions | Reports generated and displayed or exported |
| Main Flow | 1. Admin selects report type<br>2. System retrieves data<br>3. System generates report<br>4. Admin views or exports report |

**Table:2.18** Generate Financial Reports

## Manage User Roles

| Field | Description |
|---|---|
| Use Case ID | UC-09 |
| Use Case Name | Manage User Roles |
| Primary Actor | Administrator |
| Description | Allows admin to manage user roles and permissions |
| Preconditions | Administrator is logged in |
| Postconditions | User roles and access rights updated |
| Main Flow | 1. Admin selects a user<br>2. Admin assigns or updates role<br>3. System saves changes<br>4. Access control enforced |

**Table:2.19** Manage User Roles

### 2.3.3 Business Rule Documentation

Business rules define the **policies, constraints, and guidelines** that govern the operation of the Microfinance and Saving Management System (MSMS). They ensure that the system's operations are consistent with institutional policies and user expectations. Business rules are derived from stakeholder requirements, regulatory guidelines, and organizational standards. They act as a foundation for system functionality, workflow, and decision-making.

Business rules typically define **how transactions are processed**, **eligibility conditions**, **data validation**, and **reporting constraints**. Documenting these rules ensures that the system behaves predictably and aligns with organizational procedures.

#### 2.3.3.1 Key Business Rules for MSMS

| Rule ID | Business Rule | Module | Description / Constraint |
|---|---|---|---|
| BR1 | Member accounts must be unique | Member Management | Each member is identified by a unique ID number to prevent duplication of accounts. |
| BR2 | Minimum initial deposit is required | Savings Management | New members must deposit a minimum amount (e.g., 50 ETB) to activate their savings account. |
| BR3 | Loan eligibility criteria | Loan Management | Members are eligible for loans only if they have an active savings account and meet minimum savings and repayment history criteria. |
| BR4 | Maximum loan limit | Loan Management | Loans cannot exceed a predefined percentage of total member savings or a set maximum amount to mitigate risk. |
| BR5 | Repayment schedule adherence | Loan Management | Members must follow the repayment schedule; overdue repayments trigger automated alerts. |
| BR6 | Withdrawal restrictions | Savings Management | Members cannot withdraw amounts exceeding their available balance. |

| | | | |
|---|---|---|---|
| BR7 | Interest calculation | Savings & Loans | Interest on savings and loans must be calculated automatically based on predefined rates and compounding periods. |
| BR8 | Role-based access | User Access | System access is restricted based on roles: administrators have full access; members have limited access to personal information. |
| BR9 | Data integrity | All Modules | All transactions must be validated and recorded in real-time to maintain accurate balances and audit trails. |
| BR10 | Reporting accuracy | Reporting Module | Generated reports must reflect real-time data, including all savings, loans, and repayment transactions. |
| BR11 | Audit logging | All Modules | All critical operations performed by administrators must be logged to ensure accountability. |
| BR12 | System notifications | Loans & Savings | The system must automatically notify members of due payments, approvals, and important account changes. |

**Table:2.20** Key Business Rules for MSMS

### 2.3.3.2 Notes on Business Rules

Each business rule is linked to one or more **functional requirements**, ensuring that system operations align with requirements.

- ✔ Rules BR1–BR3 ensure **data consistency and operational compliance**.
- ✔ Rules BR4–BR7 enforce **financial control and correctness**, reducing risk and errors.
- ✔ Rules BR8–BR12 provide **security, transparency, and accountability**, supporting user trust and operational integrity.

**Note**: Business rules serve as a reference during **system testing, validation, and future enhancements**, ensuring consistency in functionality.

Documenting business rules is essential for defining **how the MSMS should operate** under real-world constraints. These rules ensure that the system is **reliable, accurate, and compliant with institutional policies**. By clearly specifying the constraints, conditions, and operational standards, business rules guide the development of system modules and form a foundation for modeling, design, and testing.

## Business Rules Table

| BR ID | Business Rule Description |
|---|---|
| **BR-1** | Only registered and active members are allowed to perform savings and loan transactions. |
| **BR-2** | Each member shall have a unique identification number within the system. |
| **BR-3** | A member must maintain a minimum savings balance before being eligible to apply for a loan. |
| **BR-4** | Loan applications shall be approved or rejected only by an authorized administrator. |
| **BR-5** | A member shall have only one active loan at a time. |
| **BR-6** | The maximum loan amount shall be determined based on the member's total savings. |
| **BR-7** | Loan repayment periods shall follow predefined repayment schedules. |
| **BR-8** | Loan interest rates shall be applied uniformly based on organizational policy. |
| **BR-9** | Savings withdrawals shall not exceed the available savings balance. |
| **BR-10** | Savings withdrawal requests may require administrative approval. |
| **BR-11** | All financial transactions shall be recorded and stored permanently in the system. |

| | |
|---|---|
| **BR-12** | Deleted records shall not be permanently removed but marked as inactive for audit purposes. |
| **BR-13** | System users shall only access features permitted by their assigned roles. |
| **BR-14** | System-generated reports shall reflect real-time and accurate financial data. |
| **BR-15** | Any modification to financial records shall be logged for auditing and accountability. |

**Table:2.21** Business Rules Table

### 2.3.4 User Interface Prototype

The User Interface (UI) Prototype represents the **visual and interactive design** of the Microfinance and Saving Management System (MSMS). It serves as a preliminary model for system screens, user interactions, and navigation flow. The UI prototype ensures that the system is **user-friendly, intuitive, and aligned with functional requirements** before actual implementation.

#### Purpose of the UI Prototype

- **Visual Representation:** Provides a preliminary view of how the system will appear to administrators and members.
- **User Interaction:** Illustrates how users will navigate through modules, input data, and perform transactions.
- **Validation:** Allows stakeholders to review and provide feedback on screen layouts, forms, and menu structures.
- **Design Guidance:** Serves as a reference for developers during the implementation phase, ensuring consistency in design and usability.

#### Key Features of the UI Prototype

- **Administrator-Dashboard:**
  Displays an overview of system statistics, including total members, active loans, savings summaries, and pending approvals. Provides navigation to modules such as member management, savings management, loan management, and reporting.
- **Member-Dashboard:**
  Allows members to view account balances, transaction history, loan status, and notifications. Supports basic actions such as applying for a loan or submitting savings transactions.

- **Forms and Input Screens:**
  Includes registration forms, loan application forms, deposit and withdrawal forms, and search/filter screens for reports. Validation rules are indicated to guide proper input and prevent errors.
- **Reports-Interface:**
  Shows layouts for viewing or exporting financial reports, summaries, and statements. Supports PDF/CSV export options and provides filtering by date, member, or transaction type.
- **Navigation and Usability:**
  Menus, buttons, and links are organized for ease of use. Navigation follows a logical flow to minimize user errors and improve system efficiency.

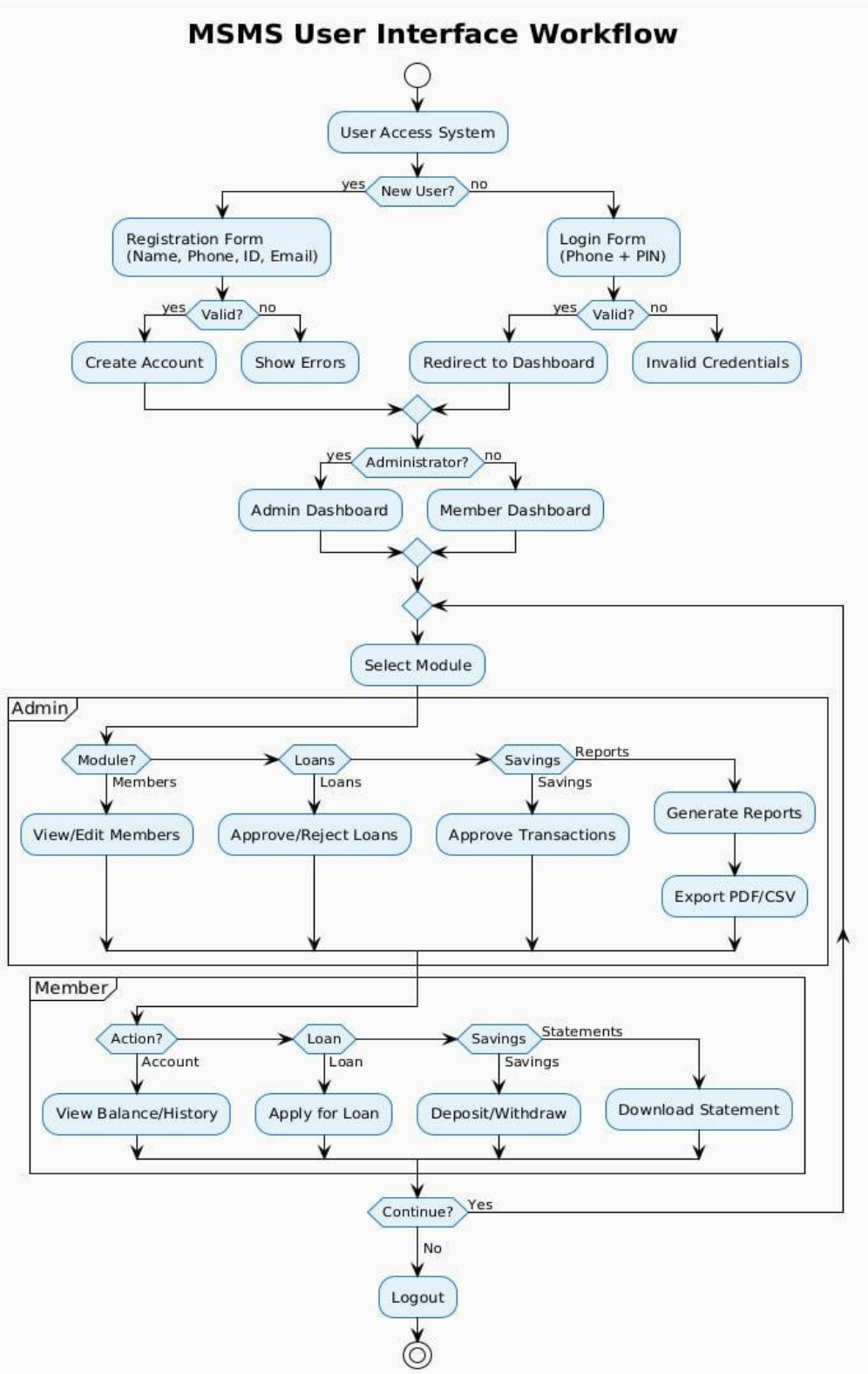



**Fig: 2.2** work flow Diagram

### 2.3.5 State Chart Diagram

The State Chart Diagram illustrates the **various states of system entities** and the transitions between these states in response to events or conditions. For the MSMS, it provides a visual representation of how key objects—such as member accounts, savings, and loans—change states during different operations (e.g., registration, deposit, withdrawal, loan approval, or repayment).

State charts help in understanding the **dynamic behavior of the system**, ensuring that all possible states and transitions are considered during design and implementation. They also support testing and validation by defining expected system behavior for different scenarios.

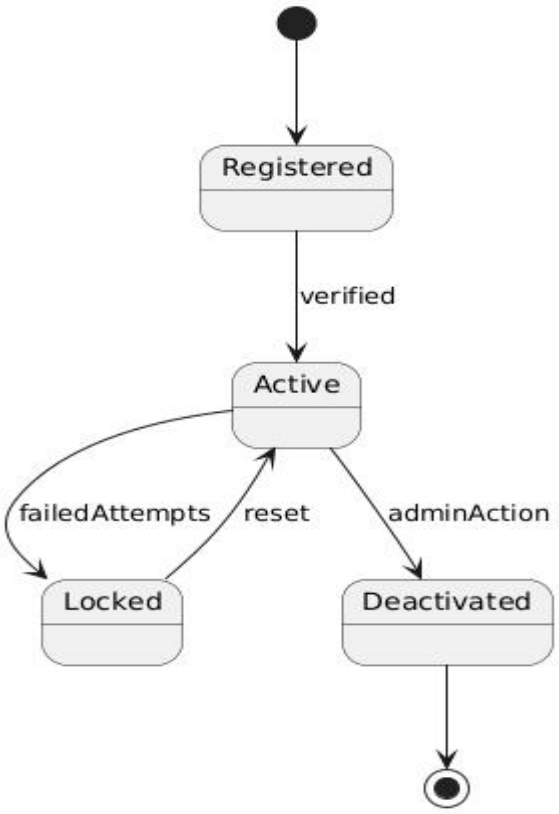



**Fig:2.3** State Chart User Account Diagram

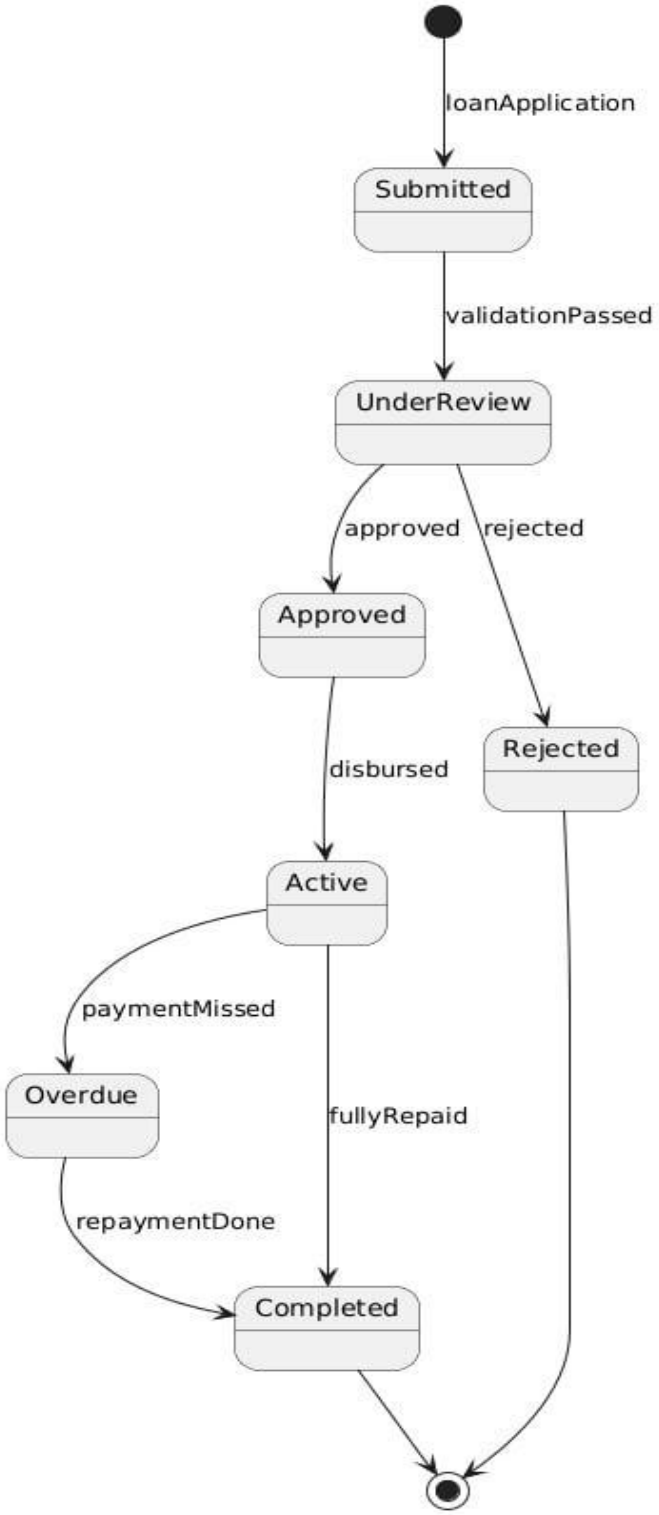



**Fig :2.4** State Chart User Loan Diagram

### 2.3.6 Activity Diagram

The Activity Diagram represents the **workflow of processes within the system**, showing the sequence of activities, decision points, and parallel operations. For the MSMS, it illustrates key operational workflows such as member registration, savings transactions, loan processing, and report generation.

Activity diagrams help in understanding the **step-by-step behavior of the system**, identifying bottlenecks, and ensuring that all process flows are clearly defined. They serve as a reference for system implementation, testing, and validation.

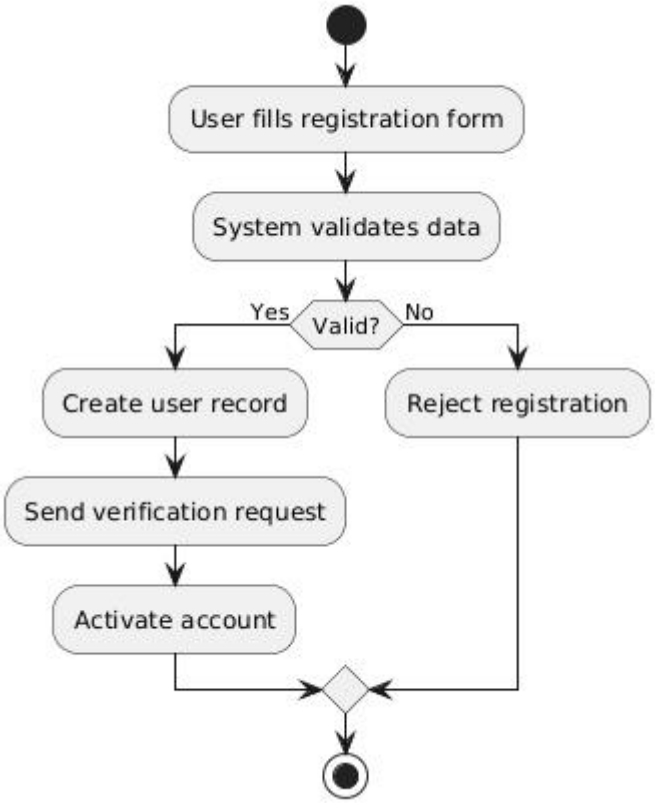



**Fig: 2.5** Register Activity Diagram

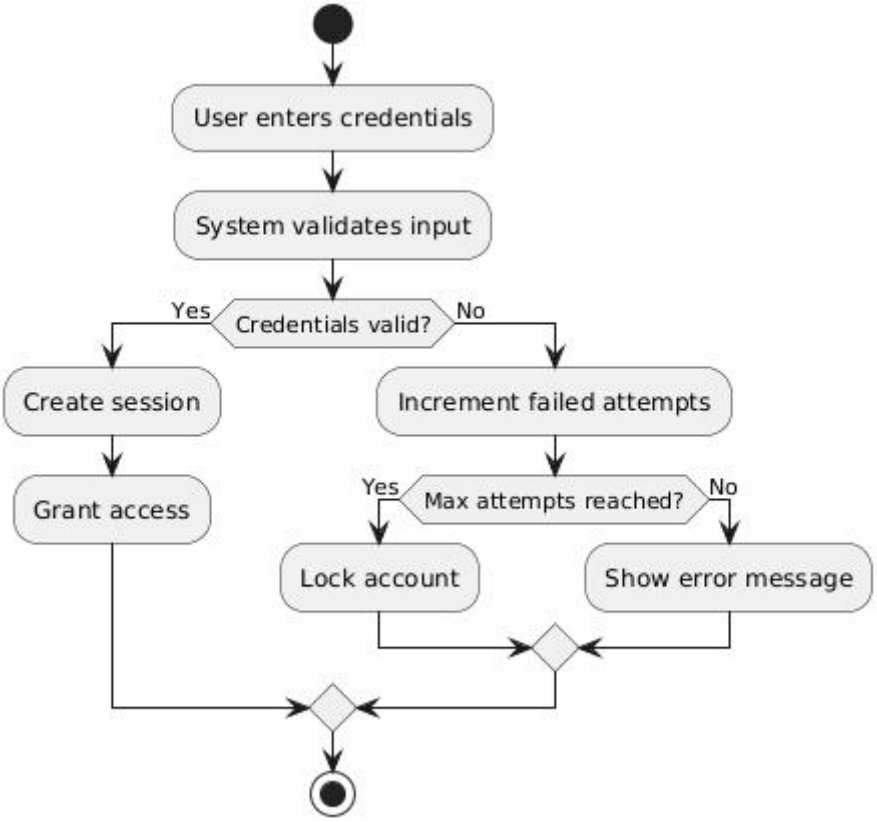



**Fig:2.6** Login Activity Diagram

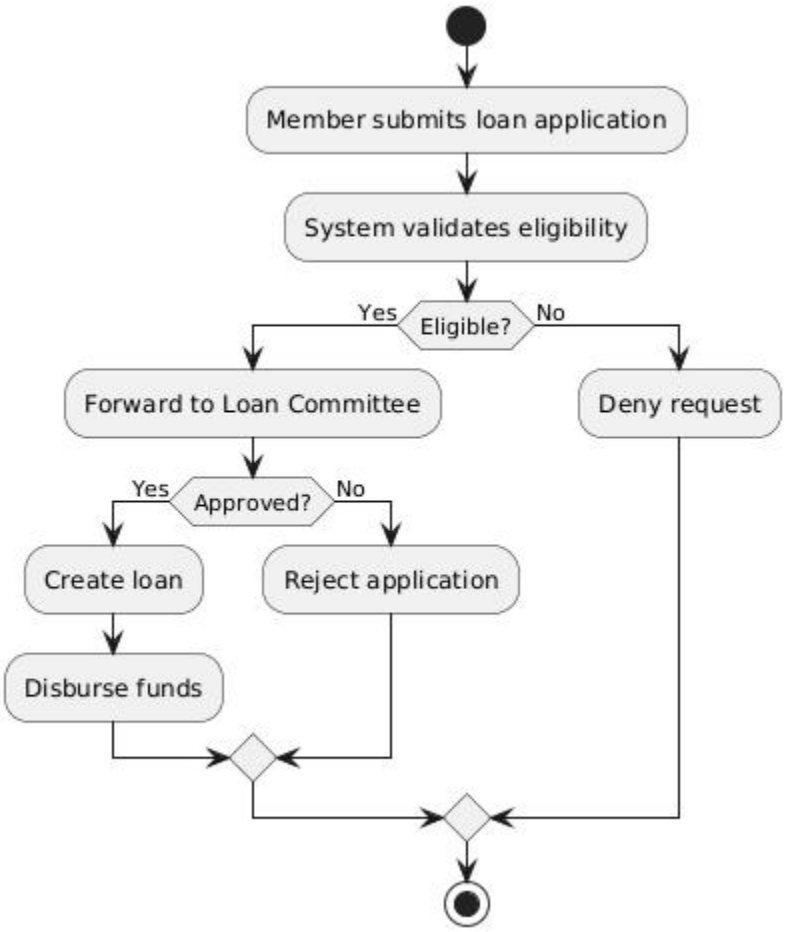



**Fig: 2.7** Loan Activity Diagram

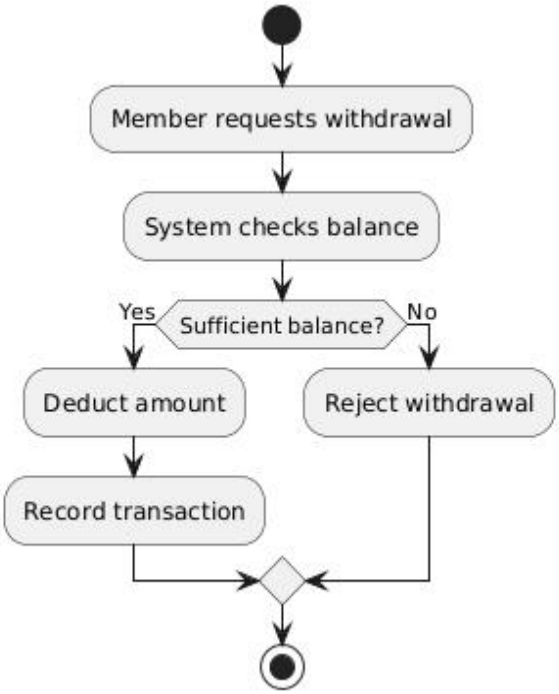



**Fig: 2.8** Withdrawal Saving Activity Diagram

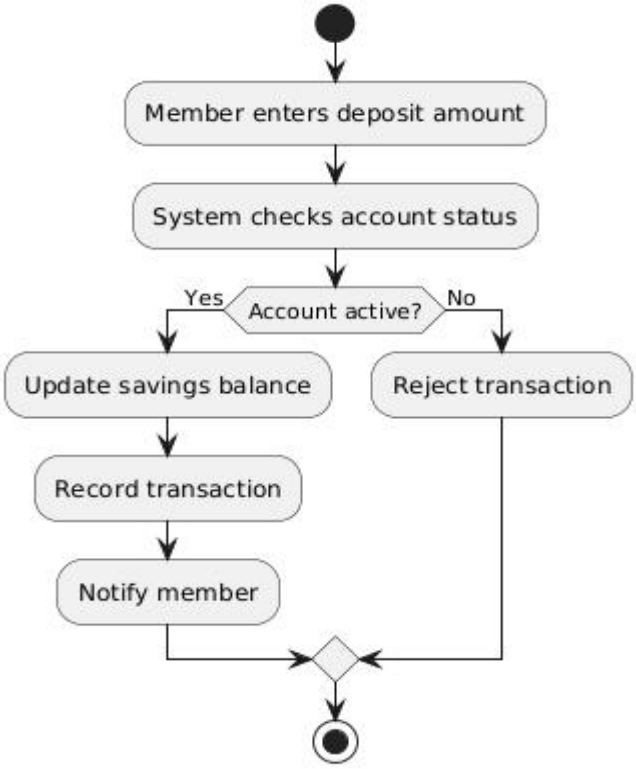



**Fig: 2.9** Saving Deposit Activity Diagram

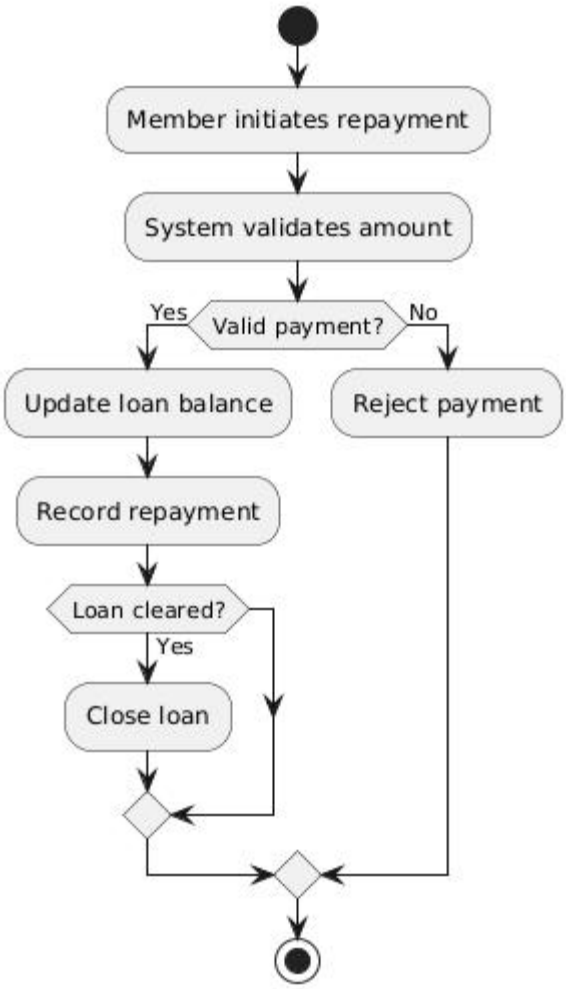



**Fig 2.10** Repay Loan  Activity Diagram

### 2.3.7 Sequence Diagram

The Sequence Diagram illustrates the **interaction between system objects over time**, showing how messages are exchanged to accomplish specific tasks. For the MSMS, sequence diagrams will represent operations such as member registration, savings deposits, loan application and approval, and report generation.
These diagrams help in understanding the **temporal order of system interactions**, ensuring that the processes follow the correct sequence and enabling accurate system implementation and testing.

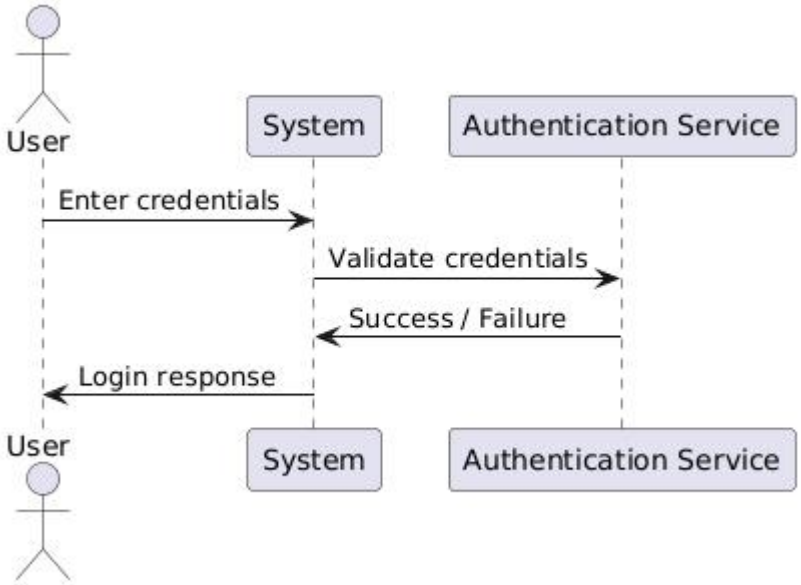



**Fig :2.11** Login Sequence Diagram

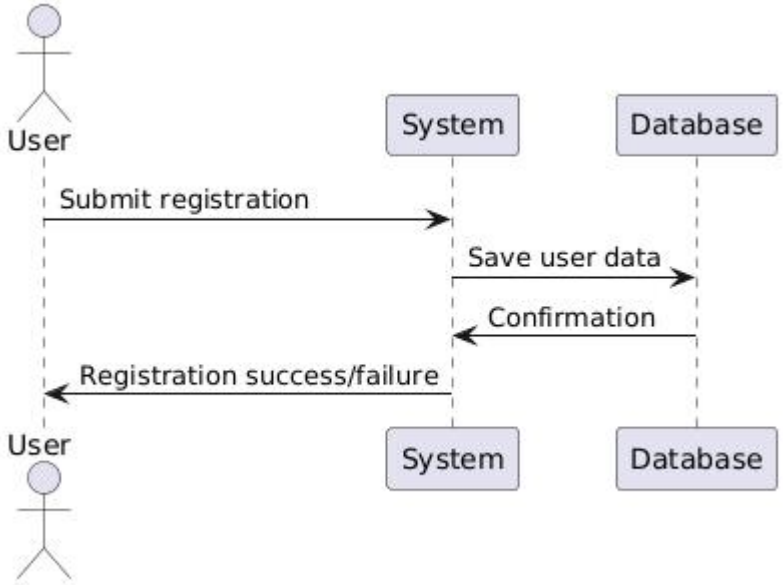



**Fig 2.12** Registration Sequence Diagram

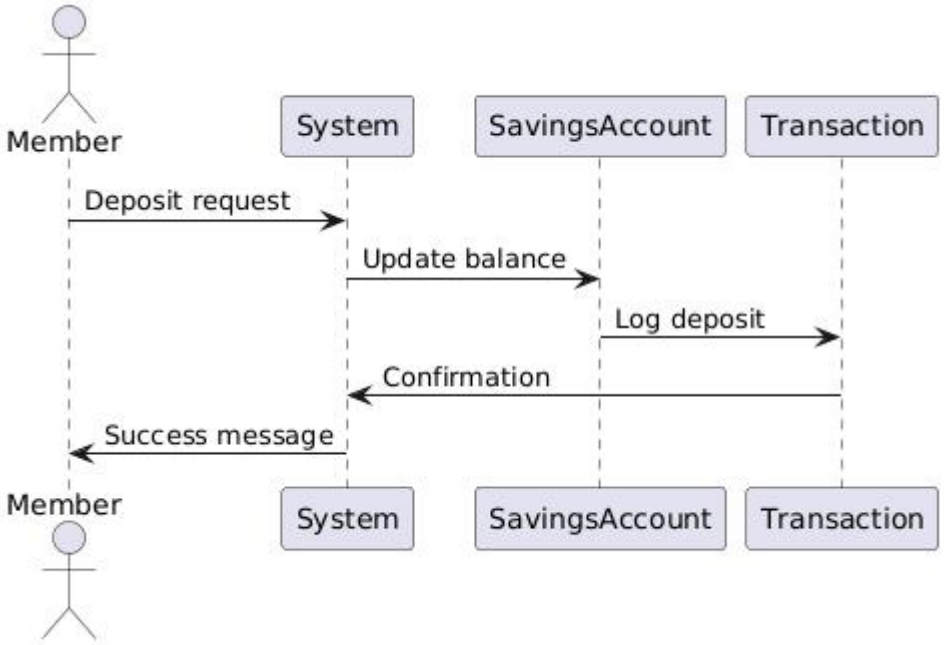



**Fig 2.13** Saving Sequence Diagram

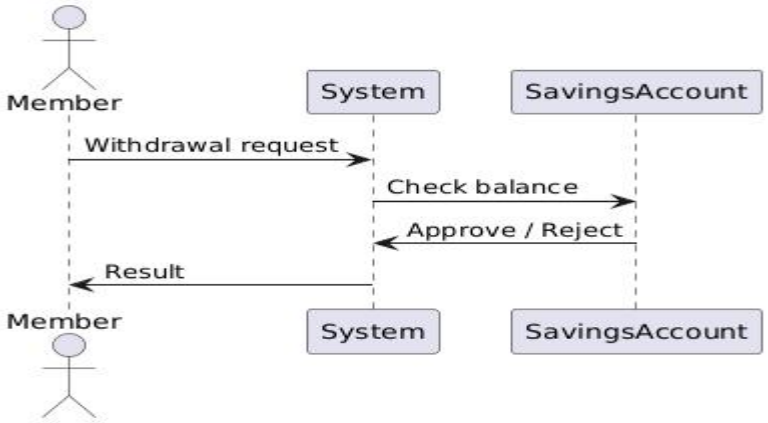



**Fig 2.14** Withdraw Savings Sequence Diagram

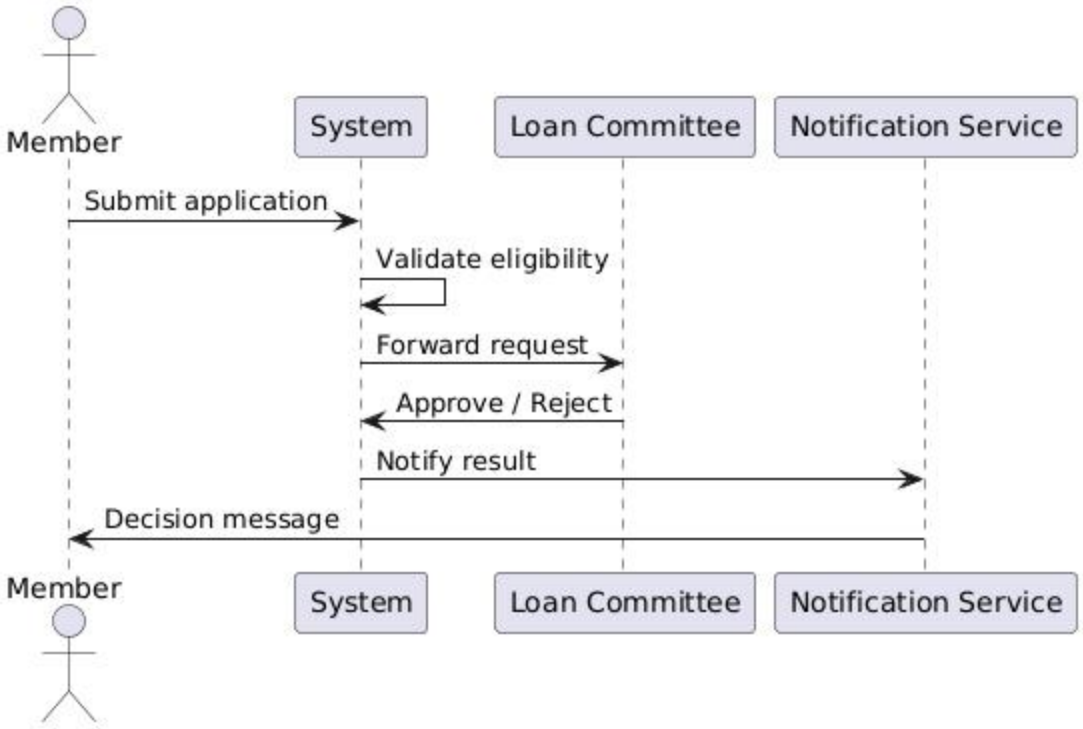



**Fig : 2.15** Loan Sequence Diagram

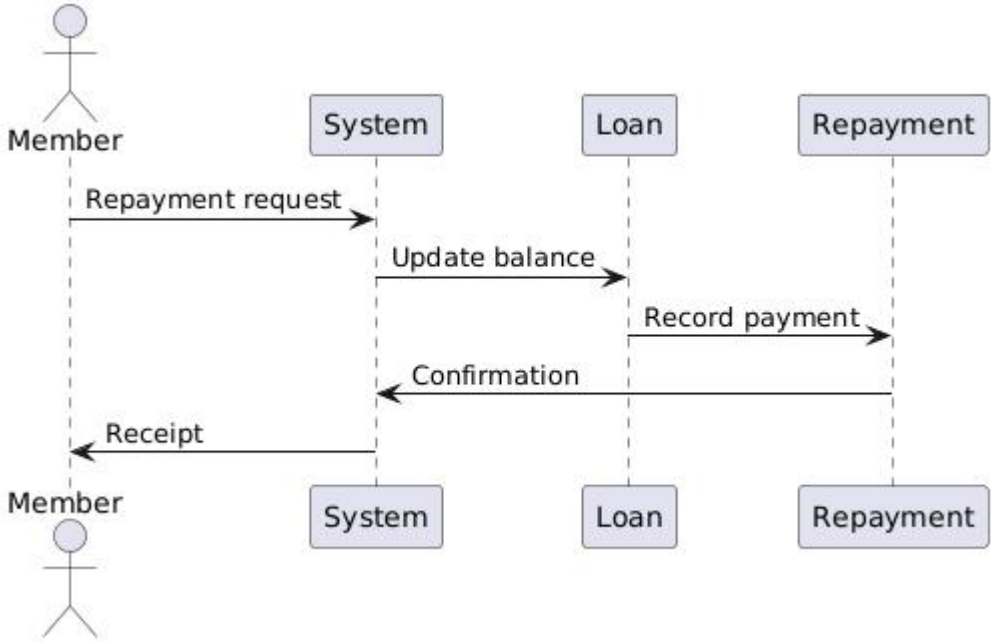



**Fig :2.16** Repay Loan Sequence Diagram

## 2.3.8 Analysis Class Model

The Analysis Class Model defines the **structure of the system in terms of classes, attributes, methods, and relationships**. For the MSMS, it represents key entities such as members, savings accounts, loans, and transactions, along with their interactions and associations.

Class diagrams help in understanding the **static structure of the system**, support object-oriented design, and serve as a foundation for implementing the system in code. They also provide clarity on data organization, relationships, and responsibilities within the system.

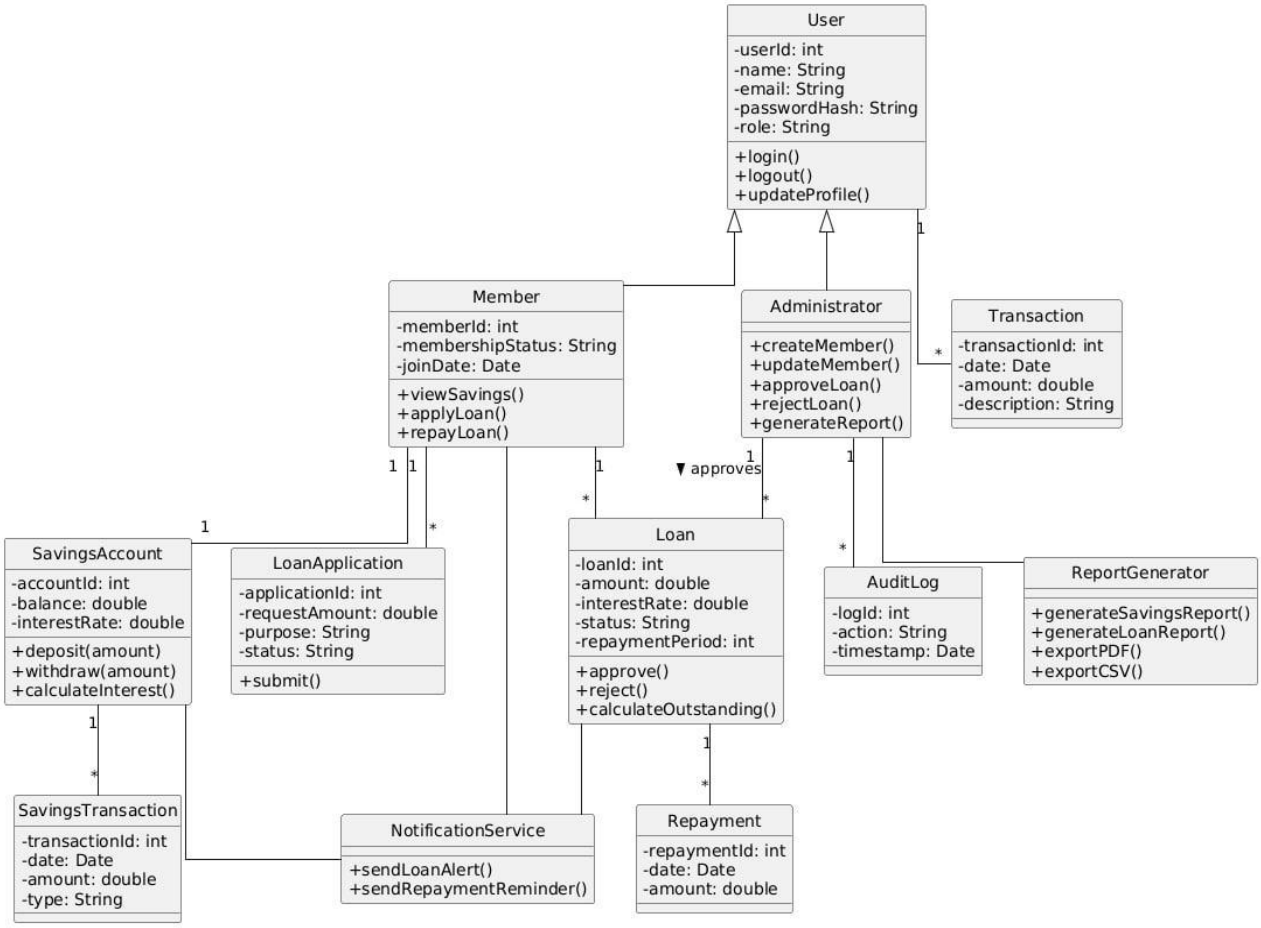



**Fig:2.17** Class Diagram

### 2.3.9 Logic Model

1.Common logic model

#### ♦ *For Login*

```
FUNCTION Login(email, password)
  GET email AND password
  VALIDATE email AND password AGAINST DATABASE
  IF VALIDATION SUCCESSFUL
    DISPLAY "Login Successful"
    REDIRECT TO HomePage
  ELSE
    DISPLAY "Invalid Credentials"
    IF ATTEMPTS > MAX_ALLOWED_ATTEMPTS
      LOCK ACCOUNT TEMPORARILY
    END IF
  END IF
END FUNCTION
```

#### ♦ *Sign-Up*

```
FUNCTION SignUp(userDetails)
  DISPLAY signUpForm
  GET userDetails (name, email, password, etc.)
  IF email OR phoneNumber ALREADY EXISTS IN DATABASE
    DISPLAY "Email/Phone already exists"
  ELSE
    VALIDATE userDetails
    IF VALIDATION SUCCESSFUL
      SAVE userDetails TO DATABASE
      DISPLAY "Sign-up Successful"
```

```
        REDIRECT TO LoginPage
      ELSE
        DISPLAY "Invalid Information"
      END IF
    END IF
END FUNCTION
```

### ♦ *For Logout*

```
FUNCTION Logout(userID)
VALIDATE userID SESSION
TERMINATE SESSION
REDIRECT TO LoginPage
DISPLAY "Logged Out Successfully"
END FUNCTION
```

## 2.Member Logic Model

### ◆ *Profile Management*

```
FUNCTION ManageProfile(memberID, updatedDetails)
  FETCH currentProfileDetails FOR memberID
  DISPLAY currentProfileDetails
  GET updatedDetails
  VALIDATE updatedDetails
  IF VALIDATION SUCCESSFUL
    UPDATE profileDetails IN DATABASE
    DISPLAY "Profile Updated Successfully"
  ELSE
```

```
    DISPLAY "Invalid Details"
  END IFEND FUNCTION
```

- ***Savings Management***

```
FUNCTION DepositSavings(memberID, amount)
  VALIDATE amount > 0
  IF VALIDATION SUCCESSFUL
    UPDATE memberBalance IN DATABASE
    RECORD transactionDetails
    DISPLAY "Deposit Successful"
  ELSE
    DISPLAY "Invalid Amount"
  END IFEND FUNCTION
FUNCTION WithdrawSavings(memberID, amount)
  FETCH currentBalance
  IF amount <= currentBalance
    UPDATE memberBalance IN DATABASE
    RECORD transactionDetails
    DISPLAY "Withdrawal Successful"
  ELSE
    DISPLAY "Insufficient Balance"
  END IFEND FUNCTION
```

- ***Loan Management***

```
FUNCTION ApplyLoan(memberID, loanDetails)
  VALIDATE loanDetails AND memberEligibility
  IF VALIDATION SUCCESSFUL
```

```
    SAVE loanApplication TO DATABASE
    DISPLAY "Loan Application Submitted"
  ELSE
    DISPLAY "Loan Application Failed"
  END IFEND FUNCTION
FUNCTION RepayLoan(memberID, loanID, amount)
  FETCH outstandingBalance FOR loanID
  IF amount <= outstandingBalance
    UPDATE loanBalance IN DATABASE
    RECORD repaymentDetails
    DISPLAY "Repayment Successful"
  ELSE
    DISPLAY "Excess Amount Entered"
  END IFEND FUNCTION
```

- ***View Reports***

```
FUNCTION GenerateReport(adminID, reportType)
  VALIDATE adminID PERMISSIONS
  FETCH data FOR reportType
  IF data EXISTS
    DISPLAY report
    OPTIONALLY EXPORT report (PDF/CSV)
  ELSE
    DISPLAY "No Data Available"
  END IFEND FUNCTION
```

## 3.Admin Logic Model

- ***Manage User Roles***

```
FUNCTION ManageUserRoles(adminID, userID, action, roleDetails)

  VALIDATE adminID PERMISSIONS

  IF PERMISSIONS GRANTED

    SWITCH(action)

      CASE "ADD":

        VALIDATE roleDetails

        ASSIGN roleDetails TO userID

        DISPLAY "Role Added Successfully"

      CASE "EDIT":

        UPDATE roleDetails FOR userID

        DISPLAY "Role Updated Successfully"

      CASE "DELETE":

        REMOVE roleDetails FROM userID

        DISPLAY "Role Removed Successfully"

      DEFAULT:

        DISPLAY "Invalid Role Action"

    END SWITCH

  ELSE

    DISPLAY "Unauthorized Access"

  END IFEND FUNCTION
```

- ***Manage System Content***

```
FUNCTION ManageSystemContent(adminID, action, contentDetails)
```

```
VALIDATE adminID PERMISSIONS
IF PERMISSIONS GRANTED
  SWITCH(action)
    CASE "ADD":
      SAVE contentDetails TO DATABASE
      DISPLAY "Content Added Successfully"
    CASE "EDIT":
      UPDATE contentDetails IN DATABASE
      DISPLAY "Content Updated Successfully"
    CASE "DELETE":
      DELETE contentDetails FROM DATABASE
      DISPLAY "Content Deleted Successfully"
    DEFAULT:
      DISPLAY "Invalid Action"
  END SWITCH
ELSE
  DISPLAY "Unauthorized Access"
END IFEND FUNCTION
```

## 2.4 Non-Functional Requirements

Non-functional requirements (NFRs) define the **quality attributes, constraints, and standards** that the system must meet. Unlike functional requirements, which specify **what the system should do**, non-functional requirements describe **how the system should perform**, ensuring reliability, efficiency, security, usability, and maintainability. For the Microfinance and Saving Management System (MSMS), non-functional requirements are crucial because the system handles **sensitive financial data** and must provide **accurate and timely services** to both administrators and members.

### 2.4.1 Performance

Performance requirements ensure that the system operates **efficiently and responds promptly** under normal and peak load conditions.

- The system must handle **simultaneous access by at least 50 members and 5 administrators** without performance degradation.
- Response time for core operations such as login, savings deposit, withdrawal, and loan processing should not exceed **3 seconds** under normal load.
- Reports and dashboards must generate and display data **within 5 seconds** for up to 1,000 transactions.
- The system should support future scalability, including the ability to handle increased users and transaction volumes without major redesign.

### 2.4.2 Security

Security is critical for protecting sensitive **financial and personal information** in MSMS.

- All user passwords must be **encrypted** in the database using secure hashing algorithms.
- Role-based access control must be implemented to ensure that only authorized users can access administrative functions or member data.
- The system must enforce **session timeouts** after periods of inactivity to prevent unauthorized access.
- Data transmission between client and server must use **secure communication protocols** (e.g., HTTPS).
- Audit logs must be maintained for all critical operations, including login attempts, transactions, and report generation, to ensure accountability.

### 2.4.3 Usability

Usability requirements ensure that the system is **user-friendly and intuitive** for both administrators and members.

- The interface should be **simple, clean, and consistent** across all modules.
- Members must be able to access their accounts, view balances, and submit loan applications with **minimal guidance**.
- Administrators should be able to manage users, transactions, and generate reports efficiently.
- The system should provide **clear error messages, prompts, and tooltips** to guide users.

- Accessibility standards should be considered, allowing the system to be used on **desktop and mobile devices** with responsive design.

### 2.4.4 Reliability

Reliability ensures that the system operates **accurately and consistently** over time.

- The system must provide **99% uptime**, excluding scheduled maintenance.
- All transactions must be **recorded accurately**, and data consistency must be maintained at all times.
- Backup and recovery procedures must be in place to restore data in case of hardware failure, software errors, or other unforeseen events.
- The system should be robust against errors, with appropriate **exception handling** to prevent crashes or data loss.

### 2.4.5 Maintainability and Scalability

- The system should be designed using **modular architecture**, allowing easy updates, enhancements, and maintenance.
- Source code and system documentation must be **well-commented and organized** for future developers.
- The system must support **scaling both vertically (upgrading hardware) and horizontally (adding more servers)** to accommodate growth in users and transactions.

***Generally:***

The non-functional requirements define the **quality standards and operational constraints** of the MSMS. By specifying performance, security, usability, reliability, and maintainability requirements, the system ensures that users can **trust, efficiently use, and scale the system**, while administrators can **manage financial operations securely and accurately**. These requirements complement the functional requirements to deliver a **robust, user-friendly, and sustainable financial management system**.

## 2.5 System Requirements

### 2.5.1 Hardware Requirements

1. **CPU:**
    - **Development:** Intel Core i5 or higher to run IDEs (VS Code), Node.js server, MySQL, and React development server efficiently.
    - **Deployment:** Cloud server or local server with at least 4-core CPU to host back-end APIs, Node.js runtime, and MySQL database.

2. **RAM:**
    - **Development:** Minimum 8 GB RAM, recommended 16 GB for running multiple services, browser tabs, and development tools simultaneously.
    - **Client Devices:** Minimum 4 GB RAM for smooth access to the web application.
3. **Storage:**
    - **Development Environment:** 256 GB SSD or higher to store source code, database files, and project dependencies.
    - **Client Devices:** Minimum 32 GB storage for local caching and smooth browser operation.
4. **Network & Connectivity:**
    - Stable internet connection for API calls, database access, and cloud deployment.
    - Local network (LAN or Wi-Fi) for internal multi-user access to the server if deployed locally.

### 2.5.2 Software Requirements

1. **Operating System:**
    - **Development:** Windows 10/11, Linux, or mac-OS to run Node.js, React, and MySQL smoothly.
    - **Client Devices:** Any OS with modern web browsers (Windows, Linux, mac-OS) to access the web application.
2. **Development Tools:**
    - **IDE:** Visual Studio Code with React, Node.js, and MySQL extensions.
    - **Languages & Frameworks:**
    - JavaScript for frontend (React) and backend (Node.js).
    - HTML, CSS for UI styling.
    - **Libraries:** Axios for API requests, Express.js for backend routing.
3. **Database:**
    - **Primary Database:** MySQL for storing user data, savings, loans, transactions, and system logs.
    - **Backup & Analytic :** Optional CSV or Excel exports for reporting purposes.
4. **Version Control:**
    - Git and GitHub for source code management, version control, and collaboration among developers.
5. **Browser Requirements:**

- ❖ Latest versions of Google Chrome or Microsoft Edge for accessing the web application.

6. **Security Tools:**
   - ❖ JWT (JSON Web Token) for secure API authentication.
   - ❖ HTTPS and encrypted database connections to protect sensitive data.
7. **Testing Tools:**
   - ❖ Jest and React Testing Library for frontend testing.
   - ❖ Postman for testing API endpoints.
   - ❖ Mocha or Jest for backend testing in Node.js

## 2.6 Key Abstraction with CRC Analysis

The **CRC (Class-Responsibility-Collaborator) Analysis** helps identify key classes, their responsibilities, and their interactions (collaborators) in the MSMS. It supports **object-oriented design and modular development** and provides a foundation for class diagrams and backend structure in Node.js.

## Key Classes and CRC Details

1. **User**

   **Responsibilities:**
   - ◆ Register, login, and logout.
   - ◆ Update profile details.
   - ◆ View account summary.

   **Collaborators:**
   - ◆ AuthController, UserController, Database (MySQL).
2. **Member**

   **Responsibilities:**
   - ◆ Manage personal savings account.
   - ◆ Apply for loans and track repayment.
   - ◆ View transaction history.

   **Collaborators:**
   - ◆ SavingsController, LoanController, User, Database.
3. **Administrator**

   **Responsibilities:**
   - ◆ Manage users (add, update, delete).
   - ◆ Approve or reject loans.
   - ◆ Generate reports.

   **Collaborators:**
   - ◆ User, LoanController, ReportGenerator, Database.

**4 Savings**

**Responsibilities:**
- ◆ Record deposits and withdrawals.
- ◆ Update account balances.

- Maintain transaction logs.

**Collaborators:**

- Member, Database, NotificationService.

**5 Loan**

**Responsibilities:**

- Handle loan applications, approvals, and disbursements.
- Track repayment schedules and overdue payments.
- Calculate interest and outstanding balance.

**Collaborators:**

- Member, Administrator, Database, NotificationService.

**6 Transaction**

**Responsibilities:**

- Record all financial operations (savings, withdrawals, loans).
- Provide audit trail for each operation.
- Support report generation.

**Collaborators:**

- Member, Savings, Loan, Database, ReportGenerator.

**7 ReportGenerator**

**Responsibilities:**

- Compile savings, loan, and transaction reports.
- Export reports in PDF or CSV.
- Provide summary analytics for administrators.

**Collaborators:**

- Administrator, Transaction, Database.

**8 Notification-service**

**Responsibilities:**

- Send alerts for loan approvals, repayment reminders, and system updates.
- Notify members of account changes or system messages.

**Collaborators:**

- Member, Administrator, Loan, Savings.

## 2.6.1 Conceptual Modeling: Class Diagram

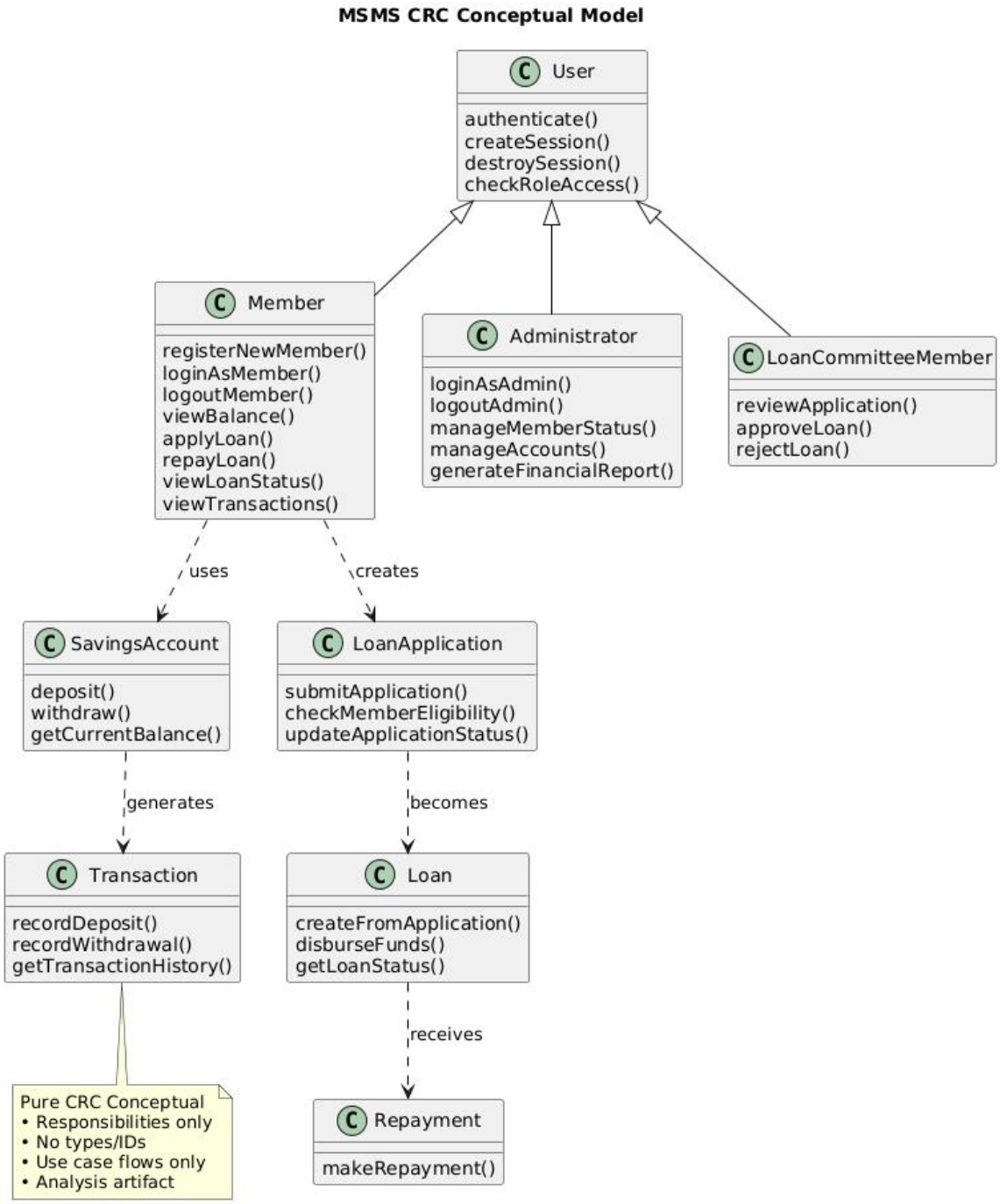



**Fig 2.18** CRC Key Model

### 2.6.2 Identifying Change Cases

Identifying potential change cases helps anticipate **future modifications, enhancements, and scalability needs** for the Microfinance and Saving Management System (MSMS). By analyzing areas where changes are likely, developers can design the system to be **flexible, maintainable, and adaptable**, reducing future development costs and minimizing disruption.

### Potential Change Cases

1. **New Financial Services:**
   - Adding new types of loans or savings products may require updates to **Loan** and **Savings** modules.
   - Impacts: Database schema adjustments, business logic updates, and changes to user interface components.
2. **Integration with External Systems:**
   - Future integration with external payment gateways, mobile wallets, or banking API s.
   - Impacts: Updates to back-end API s, security protocols, and transaction handling.
3. **User Role Expansion:**
   - Introducing new user roles such as branch managers or auditors.
   - Impacts: Modifications to **Administrator** and **User** classes, access control rules, and UI permissions.
4. **Reporting Enhancements:**
   - Adding advanced analytics, dashboards, or custom reports for administrators.
   - Impacts: Updates to **ReportGenerator**, database queries, and frontend display components.
5. **Security Enhancements:**
   - Implementation of multi-factor authentication, encryption upgrades, or compliance with new regulations.
   - Impacts: Authentication modules, database encryption layers, and user management workflows.
6. **Performance Scaling:**
   - Supporting more concurrent users as the system grows.
   - Impacts: Server upgrades, database optimization, and caching mechanisms.

By proactively identifying potential change cases, the MSMS is designed to be **scalable, maintainable, and adaptable** to future needs. Each identified change case highlights affected modules, ensuring developers can plan for **flexible architecture and minimal disruption** during system evolution.

## 2.6.3 User Interface Prototyping

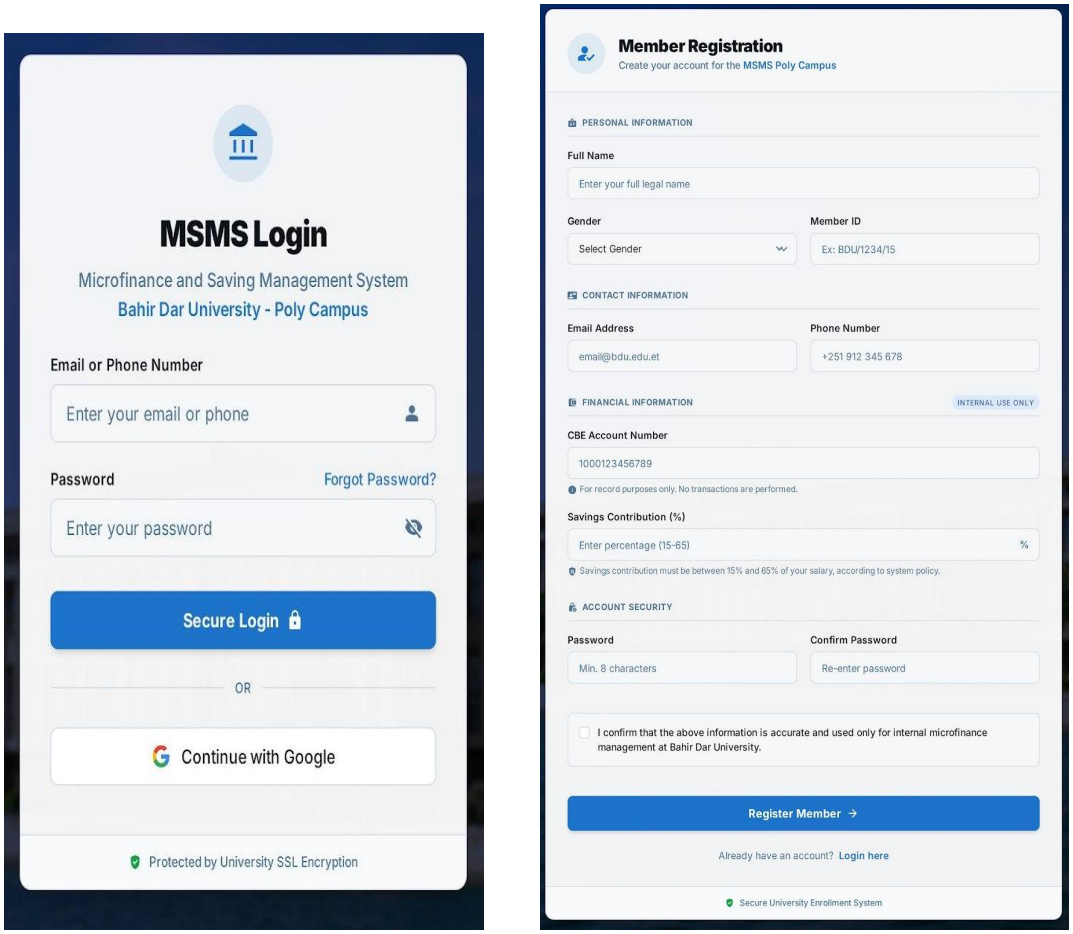



**Fig: 3.4** Login And Registration UI

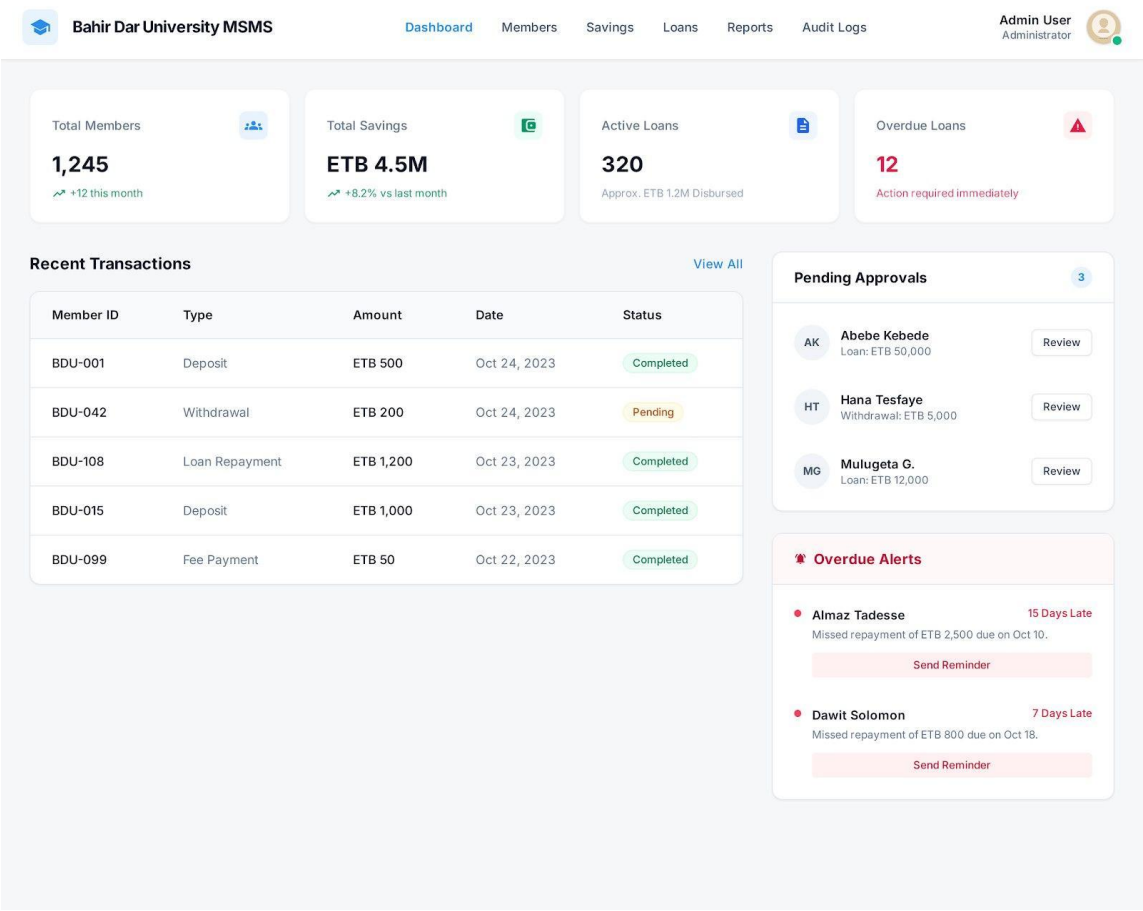
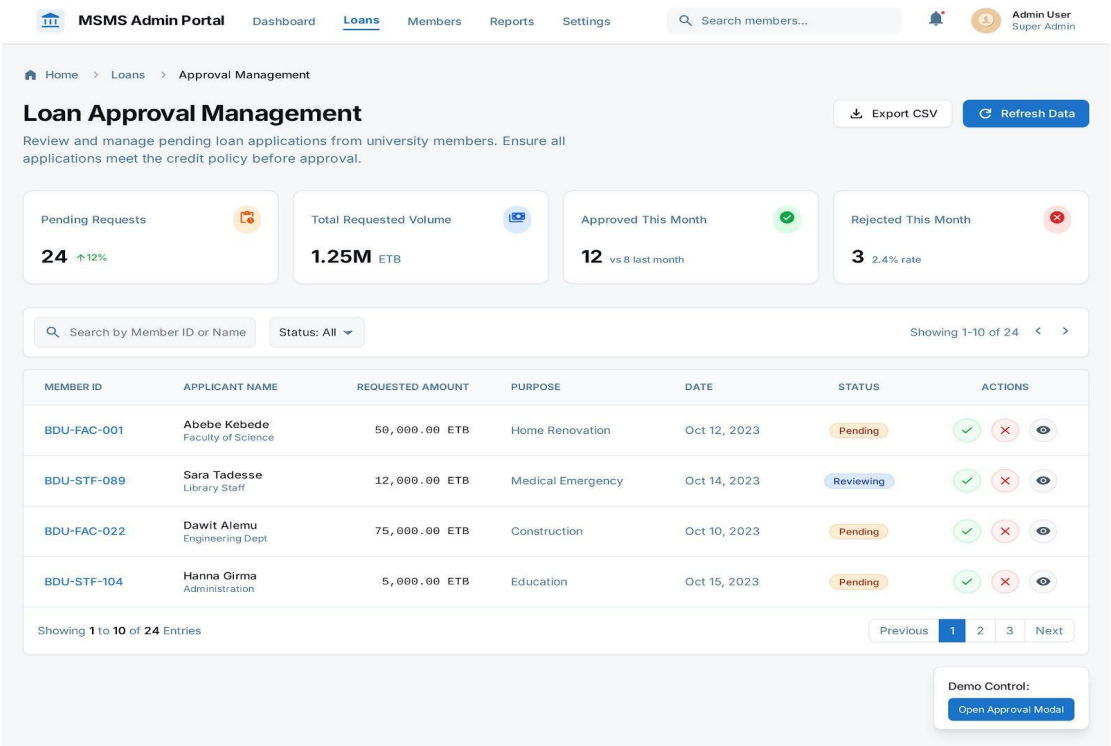



**Fig: 3.5** Admin dashboard(Admin home page) UI


MSMS Admin Portal
Dashboard
Loans
Members
Reports
Settings
Search members...
Admin User
Super Admin
Home > Loans > Approval Management
Loan Approval Management
Review and manage pending loan applications from university members. Ensure all applications meet the credit policy before approval.
Export CSV
Refresh Data
Pending Requests
24 ↑12%
Total Requested Volume
1.25M ETB
Approved This Month
12 vs 8 last month
Rejected This Month
3 2.4% rate
Search by Member ID or Name
Status: All
Showing 1-10 of 24
MEMBER ID
APPLICANT NAME
REQUESTED AMOUNT
PURPOSE
DATE
STATUS
ACTIONS
BDU-FAC-001
Abebe Kebede
Faculty of Science
50,000.00 ETB
Home Renovation
Oct 12, 2023
Pending
BDU-STF-089
Sara Tadesse
Library Staff
12,000.00 ETB
Medical Emergency
Oct 14, 2023
Reviewing
BDU-FAC-022
Dawit Alemu
Engineering Dept
75,000.00 ETB
Construction
Oct 10, 2023
Pending
BDU-STF-104
Hanna Girma
Administration
5,000.00 ETB
Education
Oct 15, 2023
Pending
Showing 1 to 10 of 24 Entries
Previous
1
2
3
Next
Demo Control:
Open Approval Modal

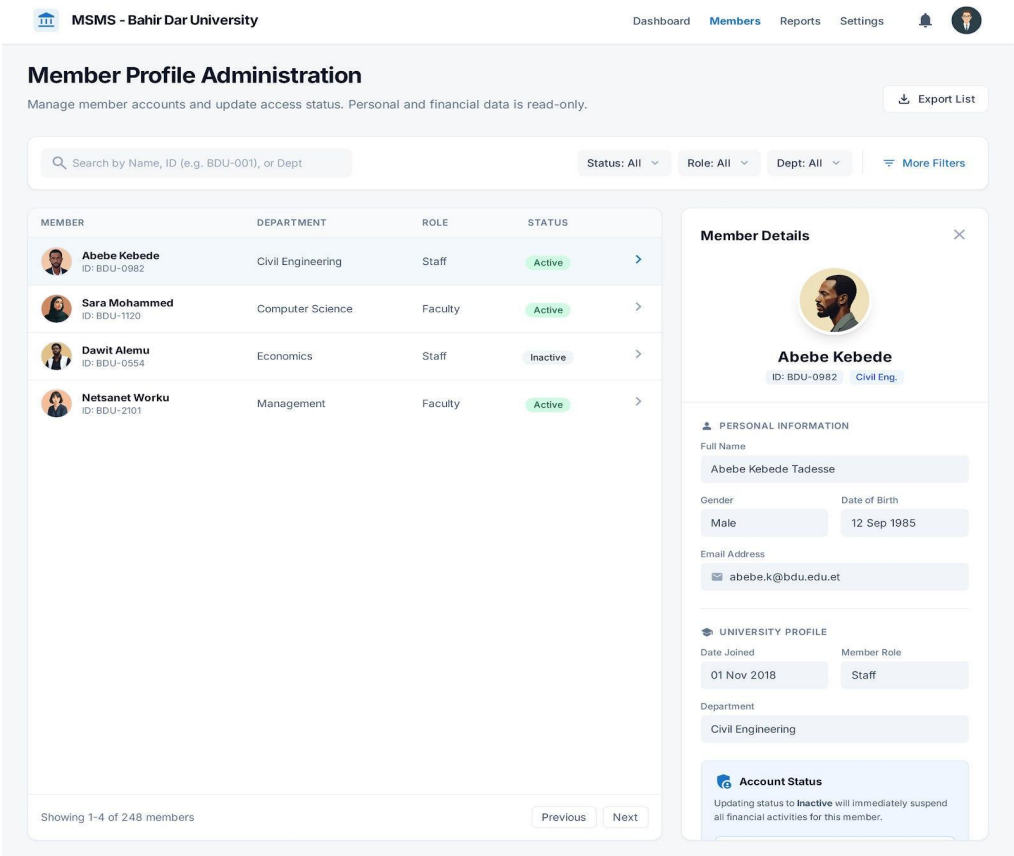

MSMS - Bahir Dar University
Dashboard
Members
Reports
Settings
Member Profile Administration
Manage member accounts and update access status. Personal and financial data is read-only.
Export List
Search by Name, ID (e.g. BDU-001), or Dept
Status: All
Role: All
Dept: All
More Filters
MEMBER
DEPARTMENT
ROLE
STATUS
Abebe Kebede
ID: BDU-0982
Civil Engineering
Staff
Active
Sara Mohammed
ID: BDU-1120
Computer Science
Faculty
Active
Dawit Alemu
ID: BDU-0554
Economics
Staff
Inactive
Netsanet Worku
ID: BDU-2101
Management
Faculty
Active
Showing 1-4 of 248 members
Previous
Next
Member Details
Abebe Kebede
ID: BDU-0982
Civil Eng.
PERSONAL INFORMATION
Full Name
Abebe Kebede Tadesse
Gender
Male
Date of Birth
12 Sep 1985
Email Address
abebe.k@bdu.edu.et
UNIVERSITY PROFILE
Date Joined
01 Nov 2018
Member Role
Staff
Department
Civil Engineering
Account Status
Updating status to Inactive will immediately suspend all financial activities for this member.

**Fig: 3.5** Other pages of Admin(Loan and User Management)

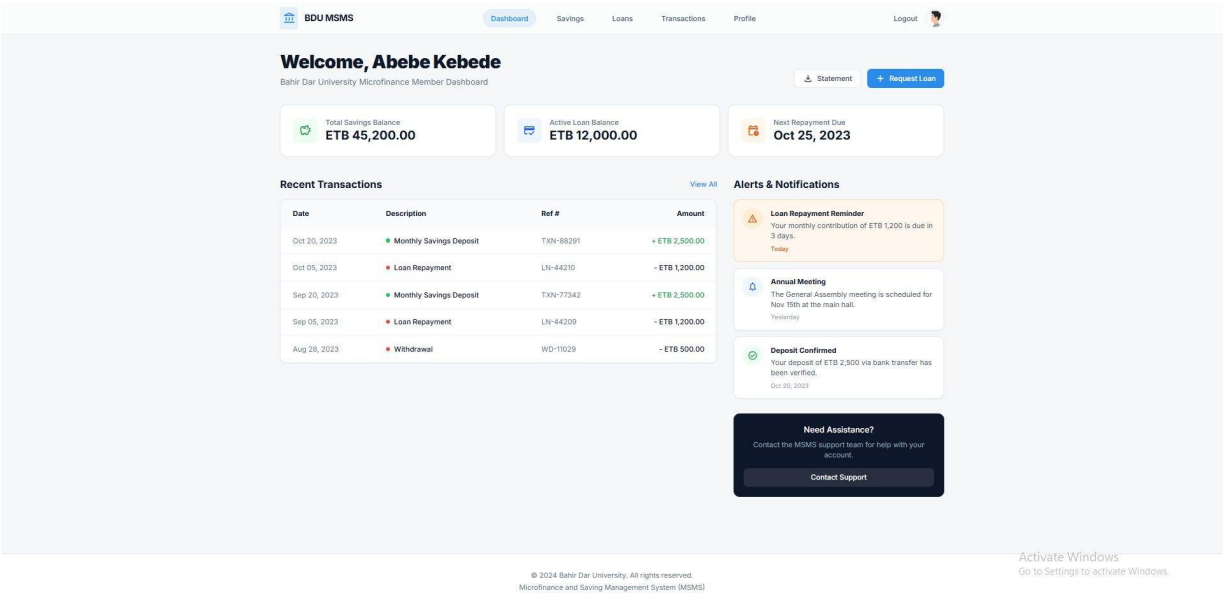



**Fig: 3.6** User Dashboard(Users Home page)

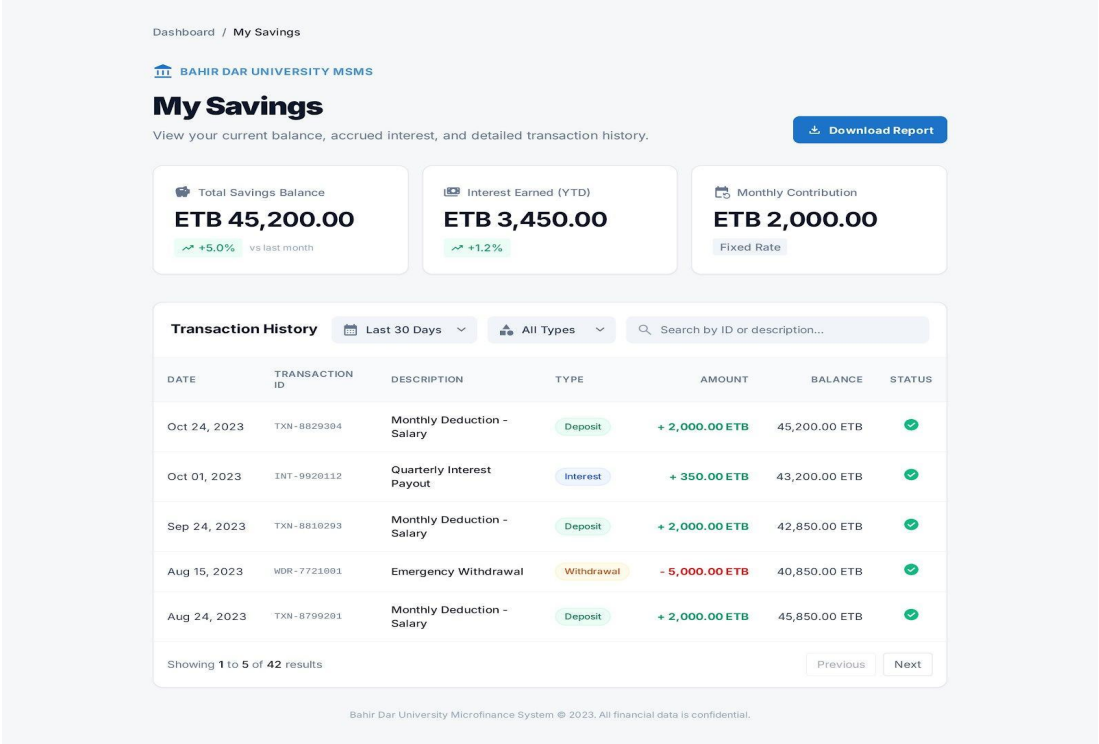


**Fig: 3.7** User Saving page()

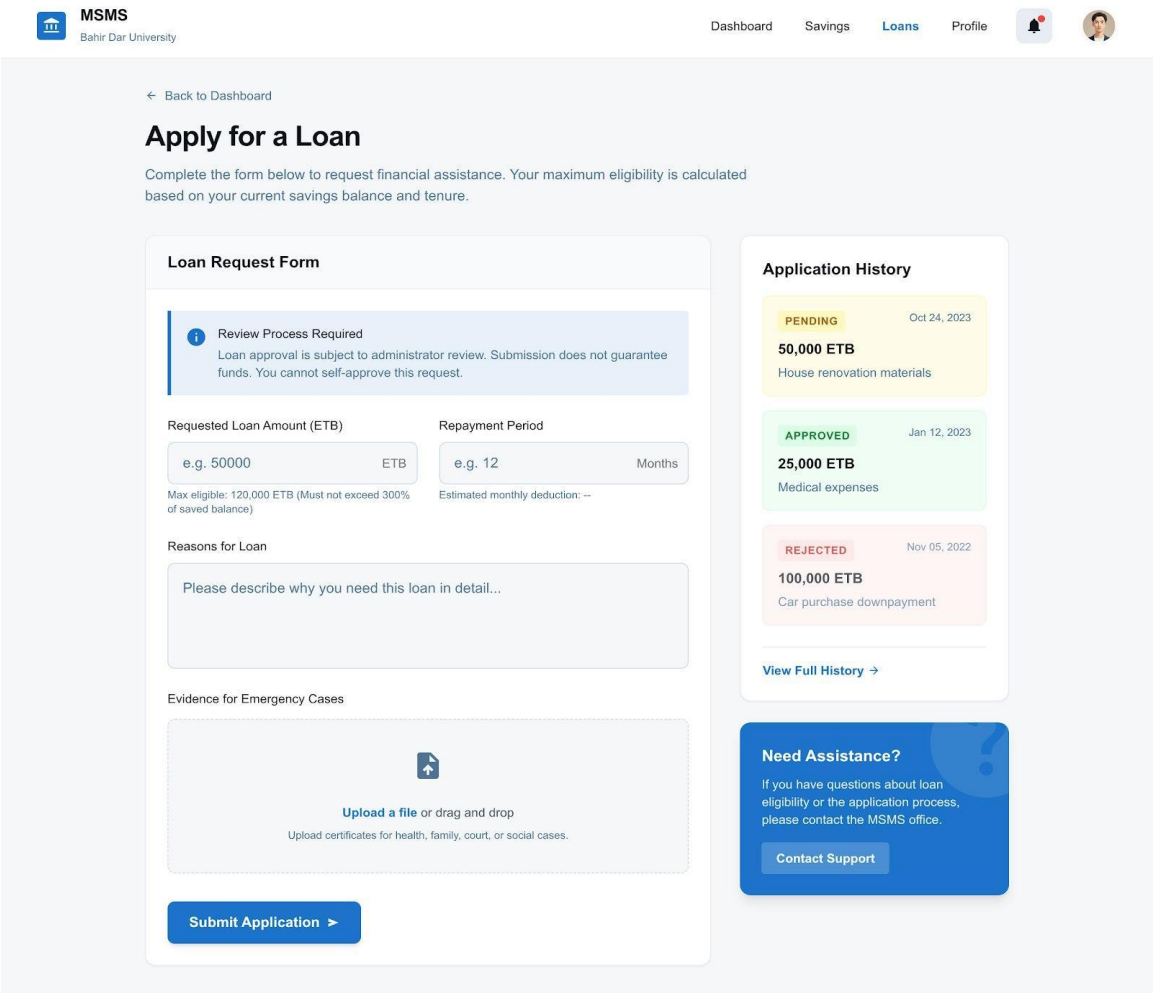



**Fig: 3.6** Loan apply page for User(Loan Request)

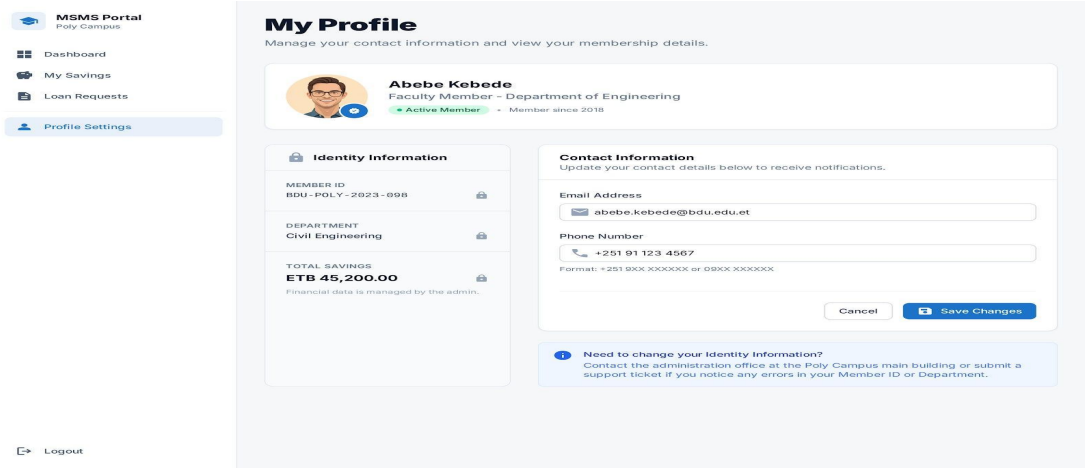



**Fig: 3.6** User Profile(Mobile view)

# Chapter Three: System Design

The system design phase is the pivotal stage where the conceptual requirements from Chapter 2 are translated into a detailed technical blueprint for the Microfinance and Saving Management System (MSMS). This chapter outlines the architectural, detailed, and user interface designs, ensuring the system is robust, secure, and aligned with the project's objectives. The design process is guided by several non-functional criteria to guarantee a high-quality final product.

## Design Goals and Criteria

| Criterion | Description | Design Implementation |
|---|---|---|
| Performance | The system must handle concurrent transactions efficiently, with response times under 2 seconds for critical operations (e.g., deposit, withdrawal, loan approval). | Optimized database queries, use of indexing, and a stateless API architecture (Node.js). |
| Dependability | The system must maintain data integrity and be available to users 99.9% of the time. | Transactional integrity via database ACID properties, automated daily backups, and error logging. |
| Security | Protection of sensitive financial and personal data | Role-Based Access Control (RBAC), JWT authentication, |

| | | |
|---|---|---|
| | from unauthorized access and malicious attacks. | password hashing (bcrypt), and input sanitization. |
| Usability (End-User) | The interface must be intuitive and easy to navigate for both members and administrators, minimizing training time. | Consistent design language (React.js/React Native), clear navigation map, and responsive design for all devices. |
| Scalability | The system must be able to support a growing user base (up to 5,000 members) and increasing transaction volume without significant performance degradation. | Layered architecture (Three-Tier), use of a relational database (MySQL) optimized for high-volume reads and writes, and potential for horizontal scaling of the application server. |
| Maintainability | The code base must be well-structured, modular, and documented to facilitate future updates and bug fixes. | Object-Oriented Programming (OOP) principles, clear separation of concerns (MVC pattern), and comprehensive API documentation. |

## 3.1 Architectural Design

The Microfinance and Saving Management System (MSMS) is designed using a **Three-Tier Client–Server Architecture**, which logically and physically separates the system into independent layers. This architectural style was selected to ensure **clear separation of concerns**, allowing each tier to evolve independently without affecting the others. By decoupling presentation, business logic, and data management, the system achieves improved **maintainability**, **scalability**, and **fault isolation**, which are critical for financial systems handling sensitive and transaction data.

Additionally, this architecture supports future system expansion, such as the integration of third-party financial services or the deployment of additional client platforms, without requiring major architectural changes.

### 3.1.1 System Architecture Overview

The MSMS architecture is composed of three primary tiers, each responsible for a distinct set of functionalities and system responsibilities.

#### Presentation Tier (Client Layer)

The presentation tier serves as the primary interaction point between the users and the system. It is implemented using a **web application developed with React.js** and a **mobile application built using React Native**. This tier is responsible for rendering user interfaces, capturing user inputs, and displaying system responses in a user-friendly manner.

The client layer performs minimal processing and does not contain business rules. Instead, it communicates with the application tier through secure HTTP-based RESTful APIs. This design ensures consistency in user experience across platforms while reducing duplication of logic.

#### Application Tier (Server Layer)

The application tier is implemented using a **Node.js runtime environment with the Express.js framework**. This tier forms the core of the system by managing business logic, request processing, and system coordination. It validates user requests, enforces business rules, manages authentication and authorization, and orchestrates interactions between the presentation tier and the data tier.

By centralizing business logic in this layer, the system ensures uniform rule enforcement and simplifies maintenance. This tier also enhances security by preventing direct access to the database from client applications.

#### Data Tier (Database Layer)

The data tier uses a **MySQL relational database management system** to store all persistent system data, including user accounts, customer records, savings transactions,

loan information, and audit logs. The relational model was chosen to maintain data integrity, support transactional consistency, and enforce relationships through constraints and keys.
This tier is exclusively accessed by the application layer, ensuring controlled data operations and minimizing the risk of unauthorized data manipulation.

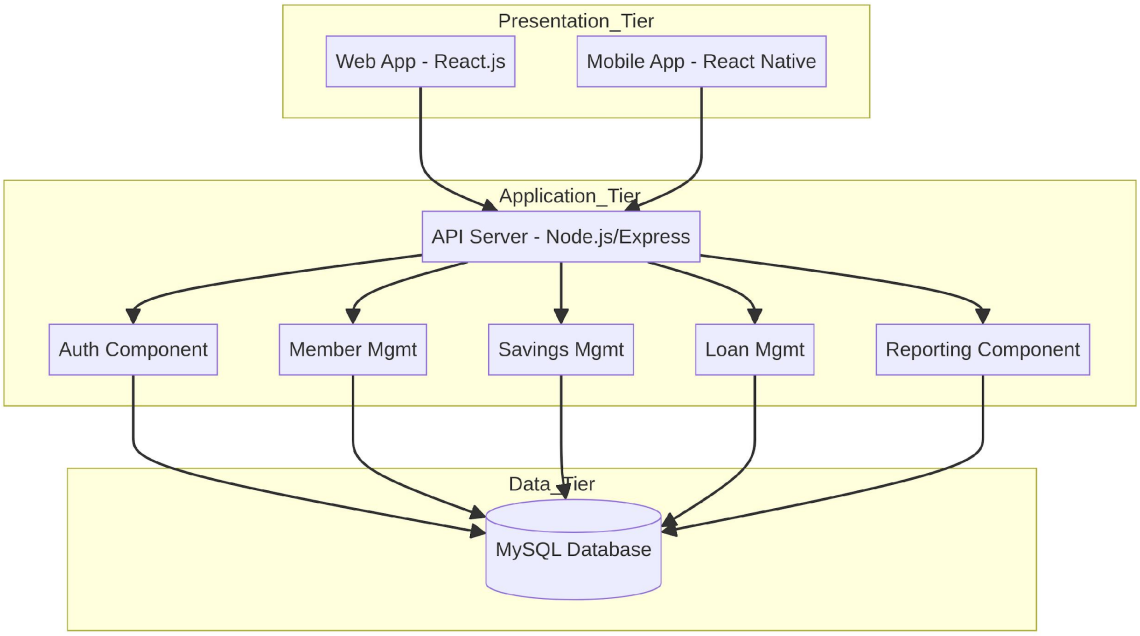



**Fig : 3.1** Application tier

### 3.1.2 Component Modeling

Component modeling decomposes the **application tier** of the Microfinance and Saving Management System (MSMS) into **cohesive, reusable, and loosely coupled components**. Each component encapsulates a specific set of responsibilities and exposes well-defined interfaces for interaction with other components. This approach improves system maintainability, enhances reusability, and simplifies future system modification or extension.

The components primarily interact through internal service calls within the application layer and access the data tier through controlled data access mechanisms. This design ensures that changes in one component have minimal impact on others, thereby reducing system complexity and risk.

| Component | Primary Services Provided | Dependencies |
|---|---|---|
| Authentication | User Registration, Login, Token Generation (JWT), Password Reset. | Data Tier (Users Table) |
| Member Management | CRUD operations for member profiles, Role assignment. | Data Tier (Members Table) |
| Savings Management | Deposit, Withdrawal, Balance Inquiry, Interest Calculation. | Data Tier (Accounts, Transactions Tables) |
| Loan Management | Loan Application, Approval/Rejection, Repayment Tracking, Interest Calculation. | Data Tier (Loans, Repayments Tables) |
| Reporting & Audit | Generate financial reports, Transaction history logs, Audit trail. | Data Tier (All Tables) |

## Component Interaction and Design Rationale

The **Authentication component** acts as a gateway for all secured operations within the system. Other components rely on it indirectly by validating user identity and access privileges before performing sensitive operations.

The **Member Management component** maintains core member data and supports role-based functionality by linking users to their respective privileges. This separation allows member-related changes to be handled independently of financial operations.

The **Savings Management** and **Loan Management components** encapsulate financial business rules, ensuring consistent processing of monetary transactions and calculations. Isolating these components improves accuracy and simplifies compliance with financial regulations.

The **Reporting & Audit component** operates across all financial modules, aggregating data to support transparency, monitoring, and decision-making. Its separation from transaction-processing components ensures that reporting operations do not interfere with core system performance.

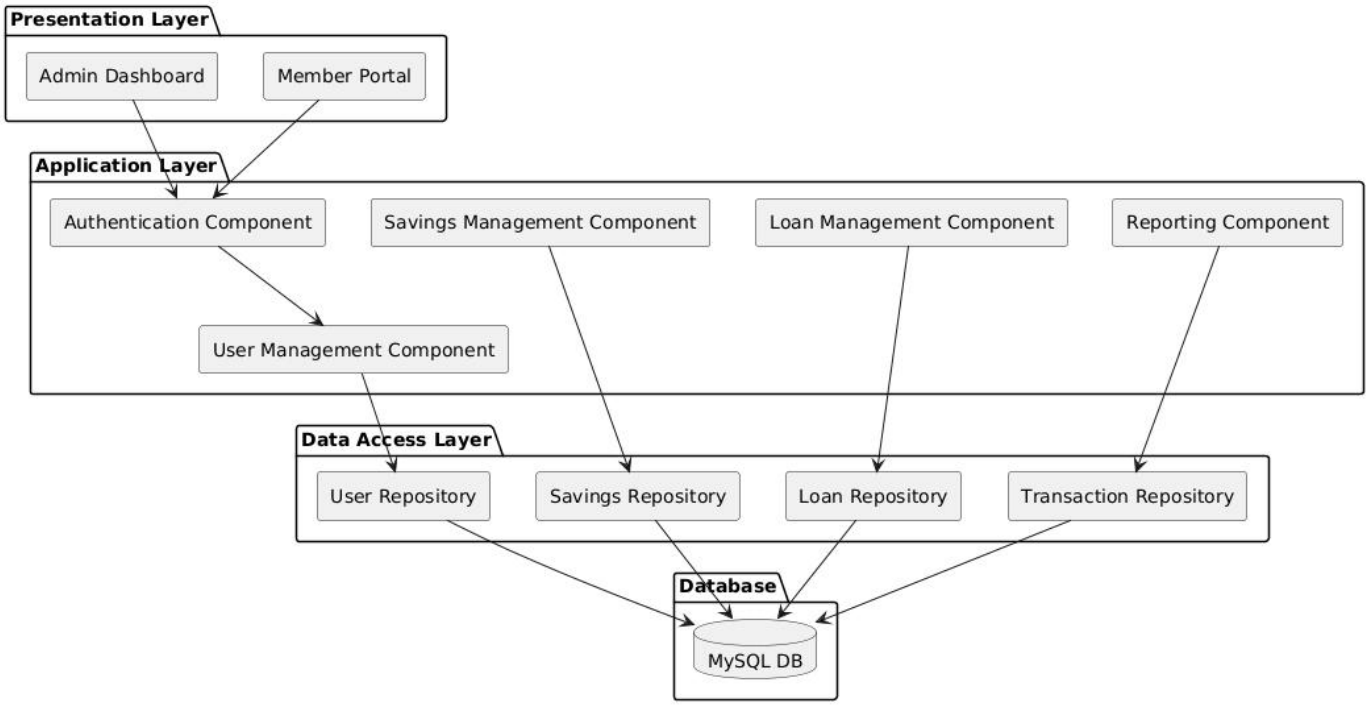



**Fig 3.2:** Component Diagram

### 3.1.3 Deployment Modeling

The deployment model illustrates the **physical distribution of software components across hardware nodes** and defines how these nodes communicate within the Microfinance and Saving Management System (MSMS). This model provides a clear view of the runtime environment and helps in understanding system scalability, security boundaries, and performance considerations.

The MSMS follows a **distributed deployment approach**, where client interfaces, application logic, and data storage are hosted on separate nodes. This separation improves

fault tolerance and allows independent scaling of system resources based on workload demand.

| Node | Description | Components Deployed | Communication Protocol |
|---|---|---|---|
| Client Devices | End-user personal computers and mobile devices running Android or iOS operating systems. | Web Application (React.js), Mobile Application (React Native). | HTTPS |
| Application Server | Cloud-hosted server responsible for executing business logic and coordinating system operations. | Node.js/Express Server (All Management Components). | HTTPS |
| Database Server | Dedicated server for persistent and transactional data storage within a secured environment. | MySQL database engine and all data tier schemas. | Secure TCP/IP (Internal Network) |

## Deployment Interaction and Design Considerations

Client devices communicate with the application server exclusively through **secure HTTPS connections**, ensuring data confidentiality and integrity during transmission. The application server exposes RESTful APIs that process client requests and enforce business rules before interacting with the database.

The **database server is isolated from direct client access** and communicates only with the application server through a secured internal network using TCP/IP. This deployment

strategy minimizes attack surfaces, improves data security, and supports compliance with financial data protection requirements.

Additionally, hosting the application server in a cloud environment allows for **horizontal and vertical scalability**, enabling the system to handle increasing numbers of users and transactions without significant architectural changes.

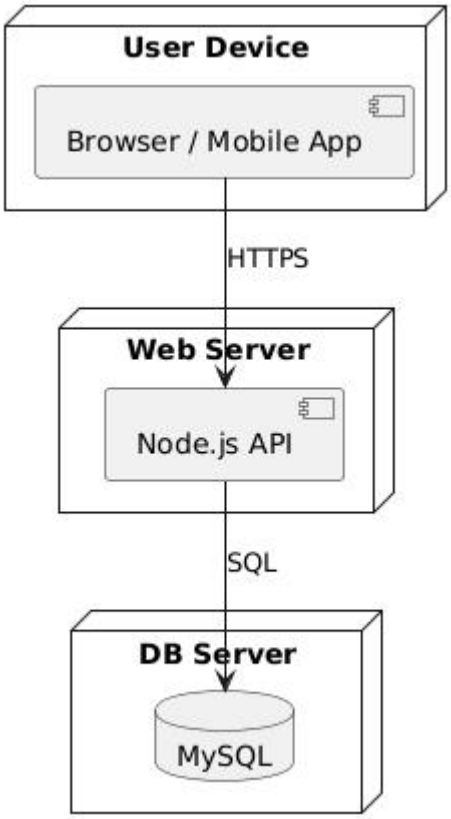



**Fig 3.3:** Deployment Diagram

## 3.2 Detailed Design

This section presents the **detailed design view** of the Microfinance and Saving Management System (MSMS). It translates the system requirements and analysis models into precise design specifications that can directly guide implementation. The detailed design emphasizes class responsibilities, data representation, and interaction patterns while ensuring consistency with the architectural and component designs defined earlier.

### 3.2.1 Design Class Model

The **Design Class Model (DCM)** is a refined version of the Analysis Class Model and incorporates **implementation-level details** such as attribute data types, method signatures, access visibility, and class-level responsibilities. This model serves as a blueprint for developers by clearly defining how system entities are structured and how they collaborate to realize system functionality.

The design classes follow object-oriented principles such as **encapsulation**, **high cohesion**, and **low coupling**, ensuring maintainable and extensible system design.

| Class | Attributes (Visibility, Name: Type) | Methods (Visibility, Name(Args): Return Type) |
|---|---|---|
| User | - private userId: int<br>- private username: string<br>- private passwordHash: string<br>- private role: enum {Admin, Member} | + public login(username: string, password: string): JWT<br>+ public logout(): void<br>+ public changePassword(oldPwd: string, newPwd: string): boolean |
| Member | - private memberId: int<br>- private fullName: string<br>- private contactInfo: string<br>- private registrationDate: Date | + public updateProfile(data: Member): boolean<br>+ public getSavingsBalance(): decimal |
| SavingsAccount | - private accountId: int<br>- private memberId: int<br>- private balance: decimal<br>- private status: enum {Active, Closed} | + public deposit(amount: decimal): boolean<br>+ public withdraw(amount: decimal): boolean<br>+ public calculateInterest(): decimal |
| Loan | - private loanId: int<br>- private memberId: int<br>- private principal: decimal | + public apply(amount: decimal, term: int): boolean<br>+ public approve(adminId: int): boolean |

| | | |
|---|---|---|
| | - private interestRate: float<br>- private termMonths: int<br>- private status: enum {Pending, Approved, Repaid} | + public recordRepayment(amount: decimal): boolean |
| Transaction | - private transactionId: int<br>- private accountId: int<br>- private type: enum {Deposit, Withdrawal, Interest}<br>- private amount: decimal<br>- private timestamp: DateTime | + public recordTransaction(): boolean<br>+ public getTransactionHistory(accountId: int): List<Transaction> |

## Class Relationships and Design Considerations

- The **User** class represents authenticated system identities and manages access control through role assignments.
- The **Member** class encapsulates domain-specific information related to microfinance customers and is logically associated with one or more financial accounts.
- A **SavingsAccount** is associated with exactly one member, while a member may own multiple savings accounts.
- The **Loan** class manages loan lifecycle states, from application to repayment completion, ensuring controlled state transitions.
- The **Transaction** class records all financial activities, providing traceability and supporting audit requirements.

Access visibility modifiers ensure that internal data remains protected, while public methods expose only controlled operations, reinforcing encapsulation.

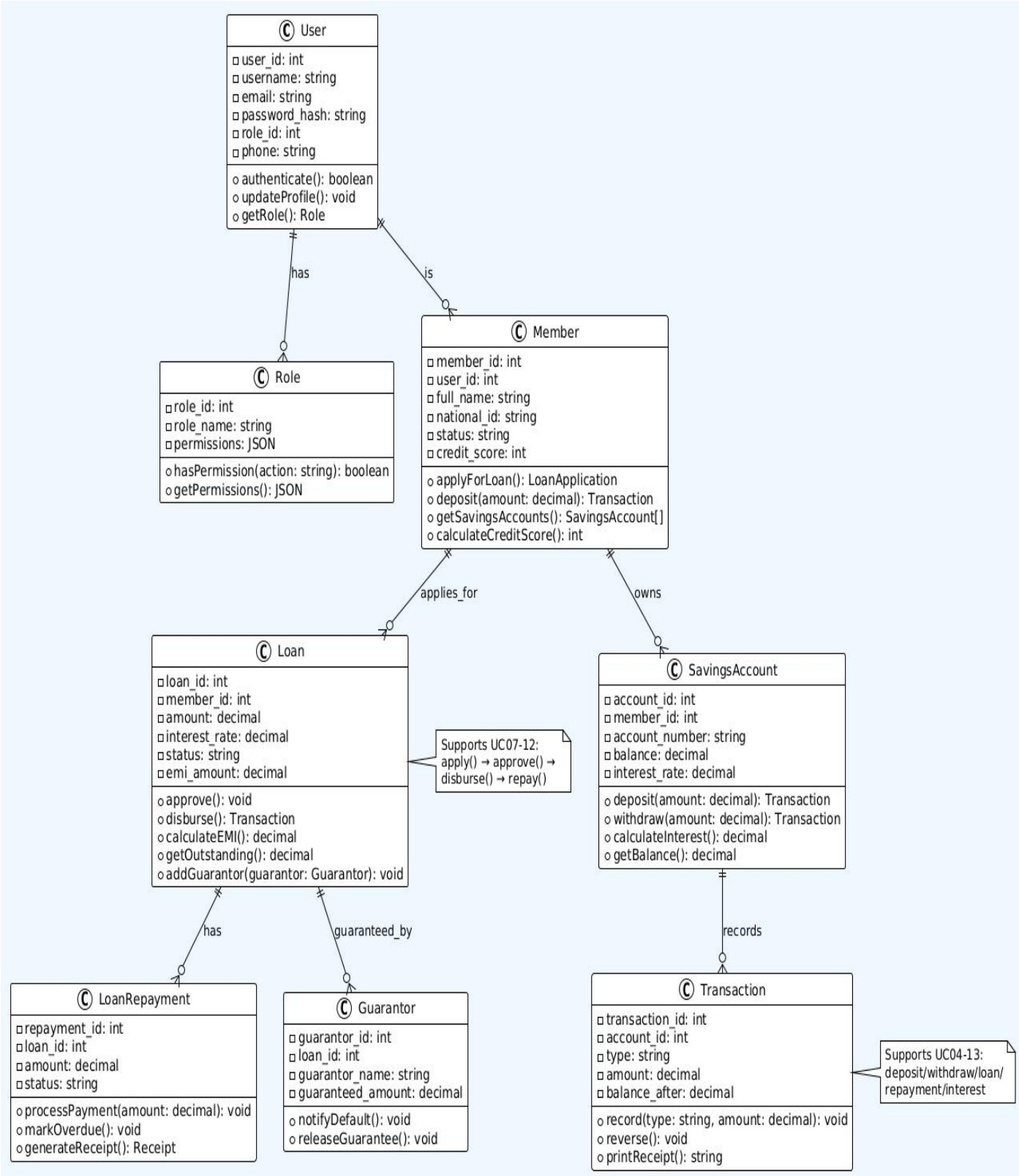



**Fig3.4:** Detailed Design Class Diagram

### 3.2.2 Persistent Model (Database Schema)

The persistent model defines how the system's data is **logically structured and physically stored** in the MySQL relational database. It ensures data integrity, consistency, and reliability by applying normalization principles, primary and foreign key constraints, and controlled relationships among entities. This model supports transactional operations such as savings deposits, withdrawals, loan processing, and audit tracking.

A relational database management system was selected to guarantee **ACID properties**, which are essential for financial systems that manage sensitive and high-value transactions.

### Entity–Relationship Diagram (ERD)

The Entity–Relationship Diagram (ERD) visually represents the core data entities and the relationships among them. It highlights cardinality, ownership, and dependency between entities such as users, members, accounts, loans, and transactions.

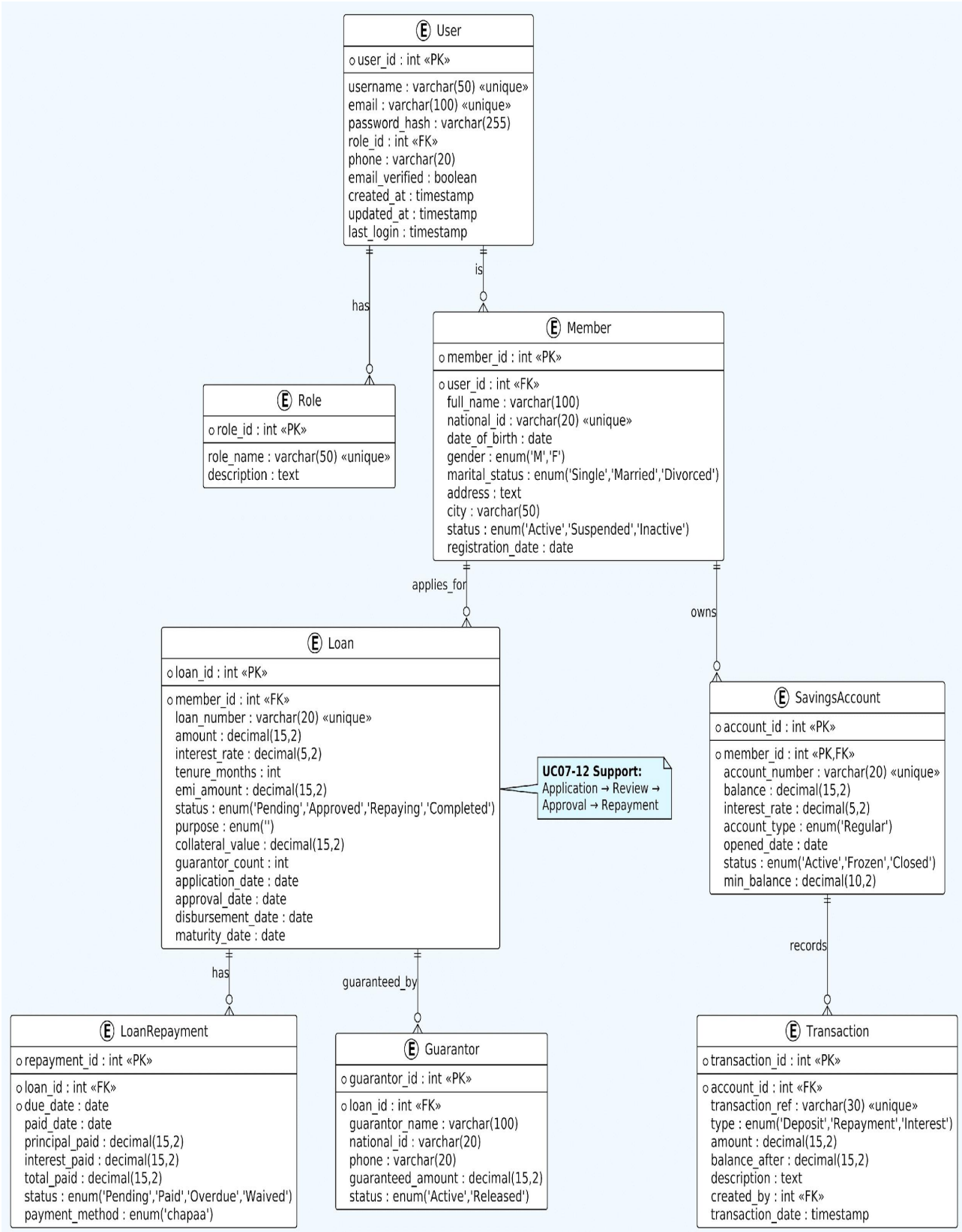



**Fig 3.5**: Detailed Entity-Relationship Diagram (ERD)

## Detailed Table Definitions

### *Users Table*

| Attribute | Data Type | Constraints | Purpose | MSMS Use Case | Validation Rules |
|---|---|---|---|---|---|
| user_id | int | PK, AUTO_INCREMENT | Unique system identifier | All authentication flows | Generated by DB |
| username | varchar(50) | UNIQUE, NOT NULL | Login identifier | UC01 User Login | 3-50 chars, alphanumeric |
| email | varchar(100) | UNIQUE, NOT NULL | Contact + recovery | Password reset, notifications | Valid email format |
| password_hash | varchar(255) | NOT NULL | Secure password storage | Authentication security | Bcrypt hash (60+ chars) |
| role_id | int | FK REFERENCES Role(role_id) | Permission level | Role-based access (UC01) | 1=Admin, 2=Member |
| phone | varchar(20) | NULL | SMS notifications | M-Pesa integration | +251 format for Ethiopia |
| email_verified | boolean | DEFAULT FALSE | Compliance check | KYC verification | Email confirmation link |
| created_at | timestamp | DEFAULT CURRENT_TIMESTAMP | Audit trail | System analytics | Automatic |
| updated_at | timestamp | ON UPDATE CURRENT_TIMESTAMP | Record tracking | Profile updates | Automatic |
| last_login | timestamp | NULL | Security monitoring | Failed login detection | Updated on login |

The **users** table manages authentication and authorization by storing login credentials and role information

.

### Role Entity

| Attribute | Data Type | Constraints | Purpose | MSMS Use Case | Validation Rules |
|---|---|---|---|---|---|
| role_id | int | PK, AUTO_INCREMENT | Role identifier | Access control | Generated by DB |
| role_name | varchar(50) | UNIQUE, NOT NULL | Human readable | Admin vs Member UI | 'Administrator', 'Member' |
| description | text | NULL | Documentation | System admin reference | Free text |

### Members Table

| Attribute | Data Type | Constraints | Purpose | MSMS Use Case | Validation Rules |
|---|---|---|---|---|---|
| member_id | int | PK, AUTO_INCREMENT | Client identifier | All member operations | Generated by DB |
| user_id | int | FK, NOT NULL | Links to auth | Single sign-on | REFERENCES User |
| full_name | varchar(100) | NOT NULL | Legal name | Loan documents | Title Case, 2+ words |
| national_id | varchar(20) | UNIQUE, NOT NULL | Government ID | KYC compliance | Ethiopian ID format |
| date_of_birth | date | NOT NULL | Age verification | Loan eligibility | >18 years |
| gender | Enum 'M','F' | NOT NULL | Demographics | Reporting | M/F only |
| marital_status | enum('Single','Married', 'Divorced') | NULL | Risk assessment | Loan scoring | Valid enum |
| address | text | NULL | Physical location | Legal notices | Free text |
| city | varchar(50) | NULL | Regional grouping | Branch assignment | Addis Ababa, etc. |
| postal_code | varchar(10) | NULL | Mail delivery | Statements | Ethiopian format |
| status | enum('Active','Suspended','Inactive') | DEFAULT 'Active' | Account state | Workflow control | State machine |

| | | | | | |
|---|---|---|---|---|---|
| registration_date | date | DEFAULT CURDATE() | Membership start | Tenure calculation | Automatic |
| kyc_verified | boolean | DEFAULT FALSE | Regulatory compliance | Loan approval | Admin approval req. |
| credit_score | int | DEFAULT 0 | Risk rating | Loan limits | 0-1000 range |

The **members** table extends the users table to store domain-specific profile information, establishing a one-to-one relationship with users.

### Saving-account Table

| Attribute | Data Type | Constraints | Purpose | MSMS Use Case | Validation Rules |
|---|---|---|---|---|---|
| account_id | int | PK, AUTO_INCREMENT | Account identifier | UC04-06 Savings | Generated by DB |
| member_id | int | PK, FK, NOT NULL | Account owner | Multi-account support | REFERENCES Member |
| account_number | varchar(20) | UNIQUE, NOT NULL | Business reference | Statements, transfers | MSMS-YYYY-XXXXX format |
| balance | decimal(15,2) | DEFAULT 0.00 | Current amount | Real-time tracking | >= min_balance |
| interest_rate | decimal(5,2) | DEFAULT 0.00 | Earnings rate | Profit calculation | 0.00-20.00% |
| account_type | Enum ('Regular') | NOT NULL | Product type | Different rules | Valid enum |
| opened_date | date | DEFAULT CURDATE() | Start date | Interest accrual | Automatic |
| status | enum('Active','Frozen','Closed') | DEFAULT 'Active' | Account state | Workflow control | State transitions |
| min_balance | decimal(10,2) | DEFAULT 0.00 | Fee trigger | Service charges | Account type rules |

This table maintains savings account information and supports balance management.

### Transactions Table

| Attribute | Data Type | Constraints | Purpose | MSMS Use Case | Validation Rules |
|---|---|---|---|---|---|
| transaction_id | int | PK, AUTO_INCREMENT | Unique transaction ID | UC04-06 Savings | Generated by DB |
| account_id | int | FK, NOT NULL | Account reference | Balance updates | REFERENCES SavingsAccount |
| transaction_ref | varchar(30) | UNIQUE, NOT NULL | Business reference | Reconciliation | MSMS-TR-YYYY-XXXXX |
| type | enum(,Repayment','Interest') | NOT NULL | Operation type | Reporting | Valid transaction type |
| amount | decimal(15,2) | NOT NULL | Transaction value | Balance calculation | >0 for deposits |
| balance_after | decimal(15,2) | NOT NULL | Running balance | Statement generation | Previous + amount |
| description | text | NULL | Human readable | Audit trail | Free text |
| created_by | int | FK, NULL | User reference | Accountability | REFERENCES User |
| transaction_date | timestamp | DEFAULT CURRENT_TIMESTAMP | Timestamp | Chronological order | Automatic |

The **transactions** table ensures complete traceability of all financial activities.

### Loans Table

| Attribute | Data Type | Constraints | Purpose | MSMS Use Case | Validation Rules |
|---|---|---|---|---|---|
| loan_id | int | PK, AUTO_INCREMENT | Unique loan identifier | UC07 Loan Application | Generated by DB |
| member_id | int | FK, NOT NULL | Borrower reference | Loan ownership | REFERENCES Member |
| loan_number | varchar(20) | UNIQUE, NOT NULL | Business reference | Statements, reports | MSMS-LN-YYYY-XXXXX |
| amount | decimal(15,2) | NOT NULL | Principal amount | Loan disbursement | >0, <= credit limit |
| interest_rate | decimal(5,2) | NOT NULL | Annual percentage | EMI calculation | 5.00-25.00% range |

| | | | | | |
|---|---|---|---|---|---|
| tenure_months | int | NOT NULL | Repayment period | EMI calculation | 3-60 months |
| emi_amount | decimal(15,2) | NOT NULL | Monthly payment | Repayment schedule | Auto-calculated |
| status | enum('Pending','Approved','Repaying','Completed') | DEFAULT 'Pending' | Loan lifecycle | Workflow control | State machine |
| purpose | enum('Business','Education','Emergency','Agriculture') | NOT NULL | Loan category | Reporting, limits | Valid business purpose |
| collateral_value | decimal(15,2) | DEFAULT 0.00 | Security value | Risk assessment | >=0 |
| guarantor_count | int | DEFAULT 0 | Group liability | Approval criteria | 0-5 guarantors |
| application_date | date | DEFAULT CURDATE() | Application timestamp | Processing SLA | Automatic |
| approval_date | date | NULL | Committee decision | UC10 Loan Approval | Admin update |
| disbursement_date | date | NULL | Funds release | UC11 Disbursement | After approval |

This table tracks **loan life-cycle** and repayment eligibility.

## LoanRepayments Table

| **Attribute** | **Data Type** | **Constraints** | **Purpose** | **MSMS Use Case** | **Validation Rules** |
|---|---|---|---|---|---|
| repayment_id | int | PK, AUTO_INCREMENT | Installment identifier | UC12 Repayment | Generated by DB |
| loan_id | int | FK, NOT NULL | Loan reference | Payment allocation | REFERENCES Loan |
| due_date | date | NOT NULL | Expected payment | Overdue detection | EMI schedule |
| paid_date | date | NULL | Actual payment | Receipt tracking | >= due_date |
| principal_paid | decimal(15,2) | NOT NULL | Principal portion | Amortization | From EMI schedule |
| interest_paid | decimal(15,2) | NOT NULL | Interest portion | Profit tracking | From EMI schedule |
| total_paid | decimal(15,2 | NOT NULL | Total | Receipt | principal + |

| | | | | | |
|---|---|---|---|---|---|
| | ) | | installment | matching | interest |
| status | enum('Pending','Paid','Overdue') | DEFAULT 'Pending' | Payment state | Collection workflow | State transitions |
| payment_method | enum('chapaa') | NOT NULL | Channel tracking | Integration logs | Valid payment method |

The **repayments** table supports tracking of partial and full loan repayments.

### Transaction Entity

| Attribute | Data Type | Constraints | Purpose | MSMS Use Case | Validation Rules |
|---|---|---|---|---|---|
| transaction_id | int | PK, AUTO_INCREMENT | Unique transaction ID | UC04-06 Savings | Generated by DB |
| account_id | int | FK, NOT NULL | Account reference | Balance updates | REFERENCES SavingsAccount |
| transaction_ref | varchar(30) | UNIQUE, NOT NULL | Business reference | Reconciliation | MSMS-TR-YYYY-XXXXX |
| type | enum('Repayment','Interest') | NOT NULL | Operation type | Reporting | Valid transaction type |
| amount | decimal(15,2) | NOT NULL | Transaction value | Balance calculation | >0 for deposits |
| balance_after | decimal(15,2) | NOT NULL | Running balance | Statement generation | Previous + amount |
| description | text | NULL | Human readable | Audit trail | Free text |
| created_by | int | FK, NULL | User reference | Accountability | REFERENCES User |
| transaction_date | timestamp | DEFAULT CURRENT_TIMESTAMP | Timestamp | Chronological order | Automatic |

This **Transaction** entity supports UC04-13: Deposit, Withdrawal, Interest, Loan Disbursement, Repayment, and Service Fees

### Guarantor Entity

| Attribute | Data Type | Constraints | Purpose | MSMS Use Case | Validation Rules |
|---|---|---|---|---|---|
| guarantor_id | int | PK, AUTO_INCREMENT | Guarantor identifier | UC08 Loan Review | Generated by DB |
| loan_id | int | FK, NOT NULL | Loan reference | Liability assignment | REFERENCES Loan |
| guarantor_name | varchar(100) | NOT NULL | Full name | Legal documents | Valid name format |
| national_id | varchar(20) | UNIQUE, NOT NULL | Government ID | KYC verification | Ethiopian ID format |
| phone | varchar(20) | NOT NULL | Contact number | Collection calls | +251 format |
| guaranteed_amount | decimal(15,2) | NOT NULL | Liability amount | Risk coverage | <= loan amount |
| status | enum('Active','Released') | DEFAULT 'Active' | Guarantor state | Loan closure | State transitions |

This **Guarantor** entity supports UC08-10: Loan Review, Approval, and group lending risk mitigation through co-liability.

### Persistence Design Considerations

- Foreign key constraints enforce **referential integrity**
- ENUM fields ensure controlled domain values
- Separation of authentication, financial, and transactional data improves maintainability
- Schema design supports **auditing, reporting, and regulatory compliance**

## 3.3 User Interface Design

The user interface (UI) design of the Microfinance and Saving Management System (MSMS) is centered on delivering a **clear, intuitive, and efficient interaction experience** for both Members and Administrators. The UI is designed to support frequent financial operations while minimizing user errors and cognitive load. Special attention is given to usability, responsiveness, and accessibility to ensure that users with varying technical proficiency can interact with the system effectively.

The interface design aligns closely with the system use cases and functional requirements defined in the earlier chapters, ensuring consistency between system behavior and user interaction.

### 3.3.1 Design Principles

The following design principles guided the development of the MSMS user interface:

1. **Consistency**
   A uniform color scheme, typography, button style, and layout structure are applied across all screens. This consistency enables users to quickly learn the system and predict interface behavior.
2. **Feedback**
   The system provides immediate visual and textual feedback for user actions such as form submission, successful transactions, and error occurrences. Examples include confirmation messages, alerts, and loading indicators.
3. **Minimalism**
   Each screen displays only essential information relevant to the current task. This reduces visual clutter and helps users focus on critical financial data and actions.
4. **Accessibility**
   The interface uses high-contrast color combinations, readable font sizes, and properly labeled form fields. These features improve usability for users with visual or motor impairments and ensure compliance with accessibility best practices.

### 3.3.2 Navigation Map

The navigation structure is designed to support **task-oriented workflows**, allowing users to move efficiently between related functions.

### Member Portal Navigation

Members access features related to personal finance management through the following navigation flow:

Login / Registration → Dashboard →Savings (Deposit, Withdrawal, Transaction History) →Loans (Loan Application, Repayment Schedule, Loan History) →Profile Settings

This structure ensures that frequently used features are accessible within minimal navigation steps.

### Administrator Portal Navigation

Administrators are provided with management and oversight functionalities through a separate navigation flow:

Login → Admin Dashboard → Member Management →Loan Approval Queue → Transaction Audit Log →Financial Reporting

This separation of portals enforces role-based access and reduces interface complexity for each user group.

### 3.3.3 UI Mockups

User interface mockups were developed to visualize the layout, content placement, and interaction flow before implementation. These mockups serve as references for both developers and stakeholders, ensuring alignment with user expectations and system requirements.

**Figure 3.6:** Mockup of the Member Login Page, illustrating authentication input fields and validation feedback.

**Figure 3.7:** Mockup of the Member Dashboard, displaying account summaries, recent transactions, and quick-action buttons.

**Figure 3.8:** Mockup of the Admin Loan Approval Page, showing pending loan requests, decision controls, and applicant details.

The mockups demonstrate how the UI supports usability, clarity, and efficient task completion for different user roles.

## 3.4 Access Control and Security

Access control and security are critical design concerns for the Microfinance and Saving Management System (MSMS) due to the sensitive nature of financial data and transactions it handles. This section describes the mechanisms employed to protect user identities, enforce controlled access to system functionalities, preserve data confidentiality and integrity, and ensure accountability for all financial operations.

The security design of MSMS is guided by the principles of **confidentiality, integrity, availability, and accountability (CIAA)**, which are essential for trustworthy financial systems.

### 3.4.1 Authentication Mechanism

MSMS uses **JSON Web Tokens (JWT)** to implement a **stateless authentication mechanism**, which is suitable for scalable, distributed, and web-based financial applications.

### Authentication Workflow

1. A user submits login credentials (username and password) through a secure login interface.
2. The application server validates the credentials against stored user records.
3. Upon successful authentication, the server generates a signed JWT containing:
    - User identifier
    - Assigned role (Member or Administrator)
    - Token issue time and expiration time
4. The token is returned to the client and stored securely.
5. For every subsequent request, the client attaches the token in the HTTP authorization header.
6. The server verifies the token signature and validity before processing the request.

### Security Benefits of JWT in MSMS

✔ **Statelessness:** No session data is stored on the server, reducing memory overhead and attack surfaces.

✔ **Scalability:** Supports horizontal scaling of application servers without session synchronization.

✔ **Tamper Resistance:** Tokens are digitally signed, preventing unauthorized modification.

✔ **Session Control:** Token expiration limits exposure in case of token compromise.

JWT-based authentication ensures that only verified users can access system services while maintaining high performance and scalability.

### 3.4.2 Authorization (Role-Based Access Control – RBAC)

After authentication, MSMS enforces **Role-Based Access Control (RBAC)** to regulate what authenticated users are allowed to do within the system. RBAC is particularly important in financial systems to prevent unauthorized financial operations and enforce organizational policies.

### Role Definition

- **Member Role:** Represents individual customers who manage their own savings and loans.
- **Administrator Role:** Represents authorized staff responsible for system oversight, approvals, and reporting.

## Authorization Matrix

| Functionality | Member Role | Administrator Role |
|---|---|---|
| View Personal Balance | Allowed | Allowed (for all members) |
| Deposit/Withdrawal | Allowed | Allowed (on behalf of member) |
| Apply for Loan | Allowed | Not Applicable |
| Approve/Reject Loan | Denied | Allowed |
| Manage Member Profiles | Denied | Allowed |
| Generate System Reports | Denied | Allowed |
| View Transaction Audit Log | Denied | Allowed |

## Authorization Enforcement

- Authorization checks are enforced at the **application server level** before executing any business logic. Each protected endpoint validates:
  - Token authenticity
  - User role
  - Requested operation permissions
- This layered approach ensures that even if a client attempts to bypass the UI, unauthorized actions are blocked at the server.

### 3.4.3 Data Security and Privacy

Given the financial nature of MSMS, strong data protection mechanisms are applied throughout the system lifecycle.

#### Password Protection and Credential Security

- User passwords are **never stored in plain text**.
- Passwords are hashed using **bcrypt**, which incorporates salting and adaptive hashing.
- This approach protects against brute-force attacks and rainbow table attacks.

#### Secure Communication

- All client–server and server–database communications are secured using **HTTPS with TLS encryption**.

- Encryption ensures confidentiality and prevents interception, tampering, or man-in-the-middle attacks.

### Input Validation and Attack Prevention

- All user inputs undergo **server-side validation**.
- Parameterized queries and ORM-based database access are used to prevent **SQL Injection**.
- Output encoding and input sanitization are applied to mitigate **Cross-Site Scripting (XSS)** attacks.

### Transaction Integrity and Non-Repudiation

- Every financial operation (deposit, withdrawal, loan repayment) is recorded in an immutable **transaction log**.
- Each transaction record includes:
  - ✔ Timestamp
  - ✔ User identity
  - ✔ Transaction type and amount
- This audit trail ensures **non-repudiation**, meaning no user can deny a transaction they initiated or approved.

### Data Privacy and Access Isolation

- Members can access only their own financial records.
- Administrators can access member data strictly for operational and reporting purposes.
- Database access privileges are restricted to the application server, preventing direct external access.

### Security Design Significance for a Financial System

The security mechanisms implemented in MSMS:

✔ Prevent unauthorized access to financial resources
✔ Protect sensitive customer information
✔ Ensure regulatory compliance and audit readiness
✔ Build trust among users and stakeholders

By combining authentication, authorization, encryption, validation, and auditing, MSMS achieves a **robust and layered security architecture**, suitable for real-world microfinance and savings management environments.

## *References*

*1. Sommerville, I. (2016). Software Engineering (10th Edition). Pearson.*
*2. Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th Edition). McGraw-Hill.*
*3. MySQL Documentation. (2024). MySQL 8.0 Reference Manual. Oracle.*
*4 . React Official Documentation. (2024). React – A JavaScript Library for Building User Interfaces. https://reactjs.org*
*5. Node.js Official Documentation. (2024). Node.js Documentation. https://nodejs.org*
*6. Git Documentation. (2024). Git – Version Control System. https://git-scm.com/doc*
*7. JWT (JSON Web Token) Documentation. (2024). JWT Authentication Standards. https://jwt.io*
*8. Bahir Dar University – Department of Computer Science. (2024). Final Year Project Guidelines & RAD Template.*
*9. Chapa Payment Gateway Documentation. (2024). https://chapa.co*

# Appendices

## Appendix A: Interview and Questionnaire

This appendix documents the instruments used during the requirement gathering phase, providing evidence of the systematic approach taken to collect and validate the system requirements.

### A.1: Interview Protocol for System Administrators

The following structured questions were used during interviews with the current system administrators and committee members to understand the existing process and pain points.

| No. | Question | Purpose |
|---|---|---|
| 1 | Can you walk me through the end-to-end process of a member making a deposit? | To map the current workflow and identify manual steps. |
| 2 | What are the three most common errors or inconsistencies you encounter with thepaper-based ledger? | To define critical non-functional requirements (e.g., accuracy, data integrity). |
| 3 | What reports do you currently generate, and how long does it take to compile them? | To define the scope and requirements for the Reporting Module (FR4.1). |
| 4 | What are the current rules for determining a member's maximum loan eligibility? | To extract and formalize the core business rules (BRs). |

## Appendix B: Detailed Test Case Suite

This suite provides a selection of detailed test cases for critical system functionalities, ensuring the system meets the defined functional requirements (FRs) and business rules. These cases are designed to be executed during the System Testing phase.

### Table B.1: Test Cases for User Authentication (Login)

| Test Case ID | Feature | Test Objective | Pre-conditions | Test Steps | Expected Result | Status |
|---|---|---|---|---|---|---|
| TC-AUT | Login | Verify successful | User "ADMIN" | 1. Navigate to the Login | System redirects to | PASS/ FAIL |

| | | | | | | |
|---|---|---|---|---|---|---|
| H-001 | | login with valid credentials. | exists with role 'Admin' and active status. | page. 2. Enter valid credentials. 3. Click "Login". | the Admin Dashboard. | |
| TC-AUTH-002 | Login | Verify failure with invalid password. | User exists with active status. | 1. Enter valid ID and incorrect password. 2. Click "Login". | Error message "Invalid credentials" is displayed. | PASS/ FAIL |

### Table B.2: Test Cases for Savings Deposit (FR2.1)

| Test Case ID | Feature | Test Objective | Pre-conditions | Test Steps | Expected Result | Status |
|---|---|---|---|---|---|---|
| TC-SAV-001 | Deposit | Verify successful deposit of a standard amount. | Admin is logged in. Member has a current balance. | 1. Select Member. 2. Enter Deposit Amount: 1000.00. 3. Click "Process Deposit". | New balance is updated correctly. A new 'Deposit' transaction record is created. | PASS/ FAIL |
| TC-SAV-002 | Deposit | Verify system handles zero or negative deposit amount. | Admin is logged in. | 1. Enter Deposit Amount: -50.00. 2. Click "Process Deposit". | System displays a validation error: "Deposit amount must be a positive value." | PASS/ FAIL |

## Appendix C: Detailed Data Dictionary

The Data Dictionary provides a comprehensive description of the data elements used within the Microfinance and Saving Management System (MSMS) database. It defines the structure, data types, constraints, and relationships for all key tables.

### Table C.1: User (Member/Admin)

| Field Name | Data Type | Constraint | Description |
|---|---|---|---|
| user_id | INT | PRIMARY KEY, Auto-Increment | Unique identifier for each user (member or administrator). |

| | | | |
|---|---|---|---|
| employee_id | VARCHAR(20) | UNIQUE, NOT NULL | Unique employee ID for the university/campus. |
| phone_number | VARCHAR(15) | UNIQUE, NOT NULL | Contact phone number. |
| role | ENUM('Member', 'Admin') | NOT NULL, Default 'Member' | User role within the system. |
| is_active | BOOLEAN | NOT NULL, Default TRUE | Status of the user account. |

### Table C.2: SavingsAccount

| Field Name | Data Type | Constraint | Description |
|---|---|---|---|
| account_id | INT | PRIMARY KEY, Auto-Increment | Unique identifier for the savings account. |
| user_id | INT | FOREIGN KEY (User.user_id) | The member who owns the account. |
| current_balance | DECIMAL(10, 2) | NOT NULL, Default 0.00 | The current monetary balance in the account (in ETB). |
| status | ENUM('Active', 'Closed', 'Suspended') | NOT NULL, Default 'Active' | Current status of the savings account. |

### Table C.3: Transaction

| Field Name | Data Type | Constraint | Description |
|---|---|---|---|
| transaction_id | INT | PRIMARY KEY, Auto-Increment | Unique identifier for the transaction. |
| account_id | INT | FOREIGN KEY (SavingsAccount.account_id) | The account involved in the transaction. |
| transaction_type | ENUM('Deposit', 'Withdrawal', 'Interest', 'Fee') | NOT NULL | Type of financial movement. |
| amount | DECIMAL(10, 2) | NOT NULL, > 0 | The monetary value of the transaction. |

| | | | |
|---|---|---|---|
| transaction_date | TIMESTAMP | NOT NULL, Default CURRENT_TIMESTAMP | Date and time the transaction occurred. |